



VISION OF THE INSTITUTE

Empower the individuals and society at large through educational excellence; sensitize them for a life dedicated to the service of fellow human beings and mother land.

MISSION OF THE INSTITUTE

To impact holistic education that enables the students to become socially responsive and useful, with roots firm on traditional and cultural values; and to hone their skills to accept challenges and respond to opportunities in a global scenario.

Lecture Notes on: Python Programming (L:T:P::3:0:2)

Course Code : CAC10

No. of Credits : 03

Contact Hours : 42 Hours (3 hours per week)

Formative Assessment Marks : 40

Summative Assessment Marks : 60

Examination Duration : 02 hours

Course Outcomes (COs):

At the end of the course, students will be able to:

- Explain the basic concepts of Python Programming.
- Demonstrate proficiency in the handling of loops and creation of functions.
- Identify the methods to create and manipulate lists, tuples and dictionaries.
- Discover the commonly used operations involving file handling.
- Interpret the concepts of Object-Oriented Programming as used in Python.
- Develop the emerging applications of relevant fields using Python.

Prepared by: Prof. Shivaprasad D L, Assistant Professor.

Department: Computer Application – B.C.A

Syllabus:

Unit - 1

Introduction to Features and Applications of Python; Python Versions; Installation of Python; Python Command Line mode and Python IDEs; Simple Python Program.

Python Basics: Identifiers; Keywords; Statements and Expressions; Variables; Operators; Precedence and Association; Data Types; Indentation; Comments; Built-in Functions- Console Input and Console Output, Type Conversions; Python Libraries; Importing Libraries with Examples.

Python Control Flow: Types of Control Flow; Control Flow Statements- if, else, elif, while loop, break, continue statements, for loop Statement; range () and exit () functions.

Exception Handling: Types of Errors; Exceptions; Exception Handling using try, except and finally.

Python Functions: Types of Functions; Function Definition- Syntax, Function Calling, Passing Parameters/arguments, the return statement; Default Parameters; Command line Arguments; Key Word Arguments; Recursive Functions; Scope and Lifetime of Variables in Functions.

Unit - 2

Strings: Creating and Storing Strings; Accessing String Characters; the str() function; Operations on Strings- Concatenation, Comparison, Slicing and Joining, Traversing; Format Specifiers; Escape Sequences; Raw and Unicode Strings; Python String Methods.

Lists: Creating Lists; Operations on Lists; Built-in Functions on Lists; Implementation of Stacks and Queues using Lists; Nested Lists.

Dictionaries: Creating Dictionaries; Operations on Dictionaries; Built-in Functions on Dictionaries; Dictionary Methods; Populating and Traversing Dictionaries.

Tuples and Sets: Creating Tuples; Operations on Tuples; Built-in Functions on Tuples; Tuple Methods; Creating Sets; Operations on Sets;

Built-in Functions on Sets; Set Methods

File Handling: File Types; Operations on Files– Create, Open, Read, Write, Close Files; File Names and Paths; Format Operator.

Unit - 3

Object Oriented Programming: Classes and Objects; Creating Classes and Objects; Constructor Method; Classes with Multiple Objects; Objects as Arguments; Objects as Return Values; Inheritance- Single and Multiple Inheritance, Multilevel and Multipath Inheritance; Encapsulation- Definition, Private Instance Variables; Polymorphism- Definition, Operator Overloading.

GU Interface: The tkinter Module; Window and Widgets; Layout Management- pack, grid and place.

Python SQLite: The SQLite3 module; SQLite Methods- connect, cursor, execute, close; Connect to Database; Create Table; Operations on Tables- Insert, Select, Update. Delete and Drop Records.

Data Analysis: NumPy- Introduction to NumPy, Array Creation using NumPy, Operations on Arrays; Pandas- Introduction to Pandas, Series and DataFrames, Creating DataFrames from Excel Sheet and .csv file, Dictionary and Tuples. Operations on DataFrames.

Data Visualisation: Introduction to Data Visualisation; Matplotlib Library; Different Types of Charts using Pyplot- Line chart, Bar chart and Histogram and Pie chart.

References:

1. Think Python How to Think Like a Computer Scientist, Allen Downey et al., 2nd Edition, Green Tea Press. Freely available online @ <https://www.greenteapress.com/thinkpython/thinkCSpy.pdf>, 2015.
2. Introduction to Python Programming, Gowrishankar S et al., CRC Press, 2019.
3. Python Data Analytics: Data Analysis and Science Using Pandas, matplotlib, and the Python Programming Language, Fabio Nelli, Apress®, 2015
4. Advance Core Python Programming, MeenuKohli, BPB Publications, 2021.
5. Core PYTHON Applications Programming, Wesley J. Chun, 3rd Edition, Prentice Hall, 2012.
6. Automate the Boring Stuff, Al Sweigart, No Starch Press, Inc, 2015.
7. Data Structures and Program Design Using Python, D Malhotra et al., Mercury Learning and Information LLC, 2021. <http://www.ibiblio.org/g2swap/byteofpython/read/> <https://docs.python.org/3/tutorial/index.html>

Unit – 1

Introduction to Features and Applications of Python; Python Versions; Installation of Python; Python Command Line mode and Python IDEs; Simple Python Program.

Python Basics: Identifiers; Keywords; Statements and Expressions; Variables; Operators; Precedence and Association; Data Types; Indentation; Comments; Built-in Functions- Console Input and Console Output, Type Conversions; Python Libraries; Importing Libraries with Examples.

Python Control Flow: Types of Control Flow; Control Flow Statements- if, else, elif, while loop, break, continue statements, for loop Statement; range () and exit () functions.

Exception Handling: Types of Errors; Exceptions; Exception Handling using try, except and finally.

Python Functions: Types of Functions; Function Definition- Syntax, Function Calling, Passing Parameters/arguments, the return statement; Default Parameters; Command line Arguments; Key Word Arguments; Recursive Functions; Scope and Lifetime of Variables in Functions.

Introduction to Python

Python is a general-purpose, high-level, interpreted programming language that supports multiple programming paradigms, including procedural, object-oriented, and functional programming, and is widely used among the developers' community. Python was mainly developed for emphasis on code readability, and its syntax allows programmers to express concepts in fewer lines of code. It was developed by Guido van Rossum during 1985- 1990. Python got its name from "*Monty Python's flying circus*". Python was released in the year 2000.

- ❖ **Python is interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it.
- ❖ **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- ❖ **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- ❖ **Python is a Beginner's Language:** Python is a great language for the beginner- level programmers and supports the development of a wide range of applications.

Why to use Python:

The following are the primary factors to use python in day-to-day life:

- ❖ **Python is object-oriented:** Structure supports such concepts as polymorphism, operation overloading and multiple inheritance.
- ❖ **Indentation:** Indentation is one of the greatest feature in python.
- ❖ **It's free (open source):** Downloading python and installing python is free and easy.
- ❖ **It's Powerful:** Dynamic typing, Built-in types and tools Library utilities, Third party utilities (e.g. Numeric, NumPy, sciPy) Automatic memory management.
- ❖ **It's Portable:** Python runs virtually every major platform used today. As long as you have a compatible python interpreter installed, python programs will run in exactly the same manner, irrespective of platform.
- ❖ **It's easy to use and learn:** No intermediate compile, Python Programs are compiled automatically to an intermediate form called byte code, which the interpreter then reads. This gives python the development speed of an interpreter without the performance loss inherent in purely interpreted languages. Structure and syntax are pretty intuitive and easy to grasp.
- ❖ **Interpreted Language:** Python is processed at runtime by python Interpreter.

- ❖ **Interactive Programming Language:** Users can interact with the python interpreter directly for writing the programs.
- ❖ **Straight forward syntax:** The formation of python syntax is simple and straight forward which also makes it popular.

Features of Python

- **Readability:** Python's syntax is clear and easy to understand.
- **Extensive Libraries:** A rich set of libraries and frameworks (e.g., NumPy, Pandas, Flask).
- **Cross-Platform:** Works on various operating systems (Windows, macOS, Linux).
- **Dynamic Typing:** Variables do not require explicit declaration.
- **Community Support:** A large community offers extensive documentation and support.

Applications of Python

- **Web Development:** Frameworks like Django and Flask.
- **Data Science:** Libraries like Pandas, NumPy, and Matplotlib.
- **Machine Learning:** Libraries like TensorFlow and Scikit-learn.
- **Automation/Scripting:** Automating repetitive tasks.
- **Game Development:** Libraries like Pygame.

Python Versions

Python has several major versions, with the most notable being:

- **Python 2.x:** Legacy version; officially discontinued in January 2020.
- **Python 3.x:** Current version with many improvements and features; recommended for all new projects.

Installation of Python

The Installation Procedure of Python:

1. **Download:** Visit the official Python website. (<https://www.python.org/downloads/>)
2. **Choose Version:** Select and download the appropriate version for your operating system.
3. **Run Installer:** Follow the installation instructions, ensuring you check the box to add Python to your PATH.
4. Verify Python Was Installed On Windows.
5. Verify Pip Was Installed.
6. Add Python Path to Environment Variables (Optional)

Python Command Line and IDEs Mode

Command Line Mode (Interactive mode)

- Open a terminal or command prompt.
- Type python or python3 to enter the interactive shell.
- You can execute Python commands directly.

Advantages:

- Python, in interactive mode, is good enough to learn, experiment or explore.
- Working in interactive mode is convenient for beginners and for testing small pieces of code.

Drawback:

- We cannot save the statements and have to retype all the statements once again to re-run them.

```

Python 2.7.13 Shell
File Edit Shell Debug Options Window Help
Python 2.7.13 (v2.7.13:a06454b1afa1, Dec 17 2016, 20:53:40) [MSC v.1500 64 bit (
AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> 2+5
7
>>> 2**6
64
>>> a=3
>>> b=6
>>> c=a-b
>>> print(c)
-3
>>> print("good morning")
good morning
>>> |

```

Integrated Development Environments (IDEs) (Script mode)

- **PyCharm:** A powerful IDE for Python with many features.
- **Visual Studio Code:** A lightweight editor with Python support through extensions.
- **Jupyter Notebook / Jupyter Lab:** Ideal for data analysis and visualization.
- **IDLE:** Comes pre-installed with Python; suitable for beginners.

Simple Python Program

Here's a simple Python program that prints "Hello, World!" is

```
# Simple Python Program
print("Hello, World!")
```

Running the Program

1. Save the code in a file named **hello.py**.
2. Open the command line and navigate to the directory where the file is saved.
3. Run the program using the command: **python hello.py**

This will output:

Hello, World!

Interactive mode	Script mode
A way of using the Python interpreter by typing commands and expressions at the prompt.	A way of using the Python interpreter to read and execute statements in a script.
Can't save and edit the code	Can save and edit the code
If we want to experiment with the code, we can use interactive mode.	If we are very clear about the code, we can use script mode.
we cannot save the statements for further use and we have to retype all the statements to re-run them.	we can save the statements for further use and we no need to retype all the statements to re-run them.
We can see the results immediately.	We can't see the code immediately.

Python Basics

Identifiers:

- **Definition:** Identifiers in Python are names used to identify variables, functions, classes, modules, and other objects in the code.
- **Rules:**
 - ❖ Must start with a letter (a-z, A-Z) or an underscore (_).
 - ❖ Can contain letters, digits (0-9), and underscores.
 - ❖ Case-sensitive (e.g., myVar and myvar are different).

Keywords:

- **Definition:** Keywords in Python are reserved words that have special meanings and serve specific purposes in the language syntax. Python keywords cannot be used as the names of variables, functions and classes or any other identifier.

Examples:

True	False	<u>None</u>	and
or	not	<u>is</u>	if
else	<u>elif</u>	<u>for</u>	<u>while</u>
<u>break</u>	<u>continue</u>	<u>pass</u>	<u>try</u>
<u>except</u>	<u>finally</u>	<u>raise</u>	<u>assert</u>
<u>def</u>	<u>return</u>	<u>lambda</u>	<u>yield</u>
<u>class</u>	<u>import</u>	from	<u>in</u>
<u>as</u>	<u>del</u>	<u>global</u>	<u>with</u>
<u>nonlocal</u>	Async	Await	

Statements and Expressions:

- **Statement:** A line of code that performs an action (e.g., assignments, function calls).
- **Expression:** A combination of values and operators that evaluates to a value (e.g., 3 + 4).

Variables:

In Python, variables are containers, which is used to store data values. Python supports various datatypes, including integers, floats, strings, and booleans. It represents the kind of value that tells what operations can be performed on a particular data. Variables are created when we assign a value to them, using the assignment operator (=).

Example:

```
Var = "Geeksforgeeks"  
print(Var)
```

Based on the data type of a variable, the interpreter allocates memory and decides what can be stored in the reserved memory. Therefore, by assigning different data types to variables, you can store integers, decimals or characters in these variables.

Rules for Python variables:

- ❖ A variable name must start with a letter or the underscore (_) character
- ❖ A variable name cannot start with a number

- ❖ A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- ❖ Variable names are case-sensitive (age, Age and AGE are three different variables)

Assigning Values to Variables:

Python variables do not need explicit declaration to reserve memory space. The declaration happens automatically when you assign a value to a variable. The equal sign (=) is used to assign values to variables.

The operand to the left of the = operator is the name of the variable and the operand to the right of the = operator is the value stored in the variable.

For example –

```
a= 100
b = 1000.0
c = "John"
print (a)
print (b)
print (c)
# An integer assignment
# A floating point
# A string
```

This produces the following result –

```
100
1000.0
John
```

Multiple Assignment:

Python allows you to assign a single value to several variables simultaneously.

For example:

```
a = b = c = 1
```

Here, an integer object is created with the value 1, and all three variables are assigned to the same memory location. You can also assign multiple objects to multiple variables.

For example:

```
a,b,c = 1,2,"MIT"
```

Here, two integer objects with values 1 and 2 are assigned to variables a and b respectively, and one string object with the value "john" is assigned to the variable c.

Output Variables:

The Python print statement is often used to output variables. Variables do not need to be declared with any particular type and can even change type after they have been set.

```
x = 5
x = "MIT "
```

```
print(x)
# x is of type int
# x is now of type str
```

Output

MIT

To combine both text and a variable, Python uses the “+” character:

Example

```
x = "awesome"
print("Python is " + x)
```

Output:

Python is awesome

You can also use the + character to add a variable to another variable:

Example

```
x = "Python is "
y = "awesome"
z = x + y
print(z)
```

Output:

Python is awesome

Input and Output

Understanding input and output operations is fundamental to Python programming. With the print() function, you can display output in various formats, while the input() function enables interaction with users by gathering input during program execution.

input ():

This function first takes the input from the user and converts it into a string. The type of the returned object always will be <class ‘str’>.

Syntax:

```
inp = input('STATEMENT')
```

Example:

```
>>>val = input("Enter your value: ")
>>>print(val)
```

Lightbox

How to Print Output in Python

Python print() function prints the message to the screen or any other standard output device.

```
>>>print("GeeksforGeeks")
```

GeeksforGeeks

```
>>>a = [1, 2, 'gfg']
```

```
>>>print(a)
```

```
[1, 2, 'gfg']
```

Expressions:

An expression is a combination of values, variables, and operators. An expression is evaluated using the assignment operator.

Examples: $Y = x + 17$

```
>>> x = 10
```

```
>>> z = x + 20
```

```
>>> z
```

```
30
```

```
>>> x = 10
```

```
>>> y = 20
```

```
>>> c = x + y
```

```
>>> c
```

```
30
```

A value all by itself is a simple expression, and so is a variable.

```
>>> y = 20
```

```
>>> y
```

```
20
```

Python also defines expressions only contain identifiers, literals, and operators. So,

Identifiers: Any name that is used to define a class, function, variable module, or object is an identifier.

Literals: These are language-independent terms in Python and should exist independently in any programming language. In Python, there are the string literals, byte literals, integer literals, floating point literals, and imaginary literals.

Operators:

In Python programming, Operators in general are used to perform operations on values and variables. These are standard symbols used for logical and arithmetic operations.

- **Arithmetic Operators:** +, -, *, /, %, ** (exponentiation), // (floor division).
- **Comparison / Relational Operators:** ==, !=, >, <, >=, <=.
- **Logical Operators:** and, or, not.
- **Assignment Operators:** =, +=, -=, *=, /=.

Operator	Token
add	+
subtract	-
multiply	*
Integer Division	/

remainder	%
Binary left shift	<<
Binary right shift	>>
and	&
or	
Less than	<
Greater than	>
Less than or equal to	<=
Greater than or equal to	>=
Check equality	==
Check not equal	!=

Generator expression:

Syntax: (compute(var) for var in iterable)

```
>>> x = (i for i in 'abc') #tuple comprehension
>>> x
<generator object <genexpr> at 0x033EEC30>
>>> print(x)
<generator object <genexpr> at 0x033EEC30>
```

You might expect this to print as ('a', 'b', 'c') but it prints as <generator object <genexpr> at 0x02AAD710> The result of a tuple comprehension is not a tuple: it is actually a generator. The only thing that you need to know now about a generator now is that you can iterate over it, but ONLY ONCE.

Conditional expression:

Syntax: true_value if Condition else false_value

```
>>> x = "1" if True else "2"
>>> x
'1'
```

Statements:

A statement is an instruction that the Python interpreter can execute. We have normally two basic statements, the assignment statement and the print statement. Some other kinds of statements that are if statements, while statements, and for statements generally called as control flows.

Examples:

An assignment statement creates new variables and gives them values:

```
x=10
```

expression is evaluated.

For example, $x = 7 + 3 * 2$; here, x is assigned 13, not 20 because operator * has higher precedence than +, so it first multiplies $3*2$ and then adds into 7.

Example 1:

```
>>> 3+4*2
11
```

Multiplication gets evaluated before the addition operation

```
>>> (10+10)*2
```

Parentheses () overriding the precedence of the arithmetic operators

Example 2:

```
a = 20
b = 10
c = 15
d = 5
e = 0
e = (a + b) * c / d      # (30 * 15) / 5
print("Value of (a + b) * c / d is ", e)
e = ((a + b) * c) / d    # (30 * 15) / 5
print("Value of ((a + b) * c) / d is ", e)
e = (a + b) * (c / d);   # (30) * (15/5)
print("Value of (a + b) * (c / d) is ", e)
e = a + (b * c) / d;     # 20 + (150/5)
print("Value of a + (b * c) / d is ", e)
```

Output:

```
C:/Users/MIT/AppData/Local/Programs/Python/Python38-32/pyyy/opprec.py
Value of (a + b) * c / d is 90.0
Value of ((a + b) * c) / d is 90.0
Value of (a + b) * (c / d) is 90.0
Value of a + (b * c) / d is 50.0
```

Precedence and Association:

- **Precedence:** Determines the order of operations in expressions.
- **Association:** Defines the direction in which operations are performed (left-to-right or right-to-left).

Example:

```
result = 3 + 4 * 2 # result is 11 (multiplication has higher precedence)
```

Data Types:

Python Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data. The following are the standard or built-in data types in Python:

- **Numeric** – int, float, complex
- **Sequence Type** – string, list, tuple
- **Mapping Type** – dict
- **Boolean** – bool
- **Set Type** – set, frozenset
- **Binary Types** – bytes, bytearray, memoryview

```
a = 5
print(type(a))

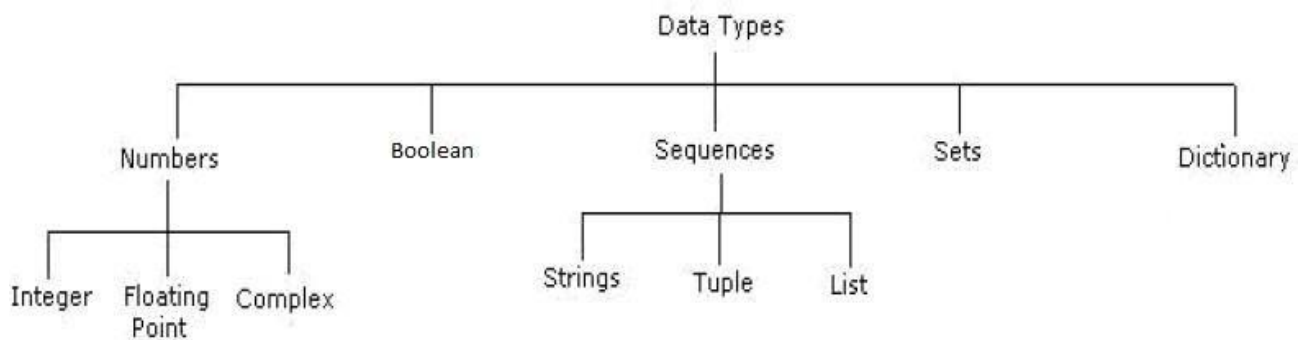
b = 5.0
print(type(b))
```

```
c = 2 + 4j
print(type(c))
```

Output:

```
<class 'int'>
<class 'float'>
<class 'complex'>
```

The data stored in memory can be of many types. For example, a student roll number is stored as a numeric value and his or her address is stored as alphanumeric characters. Python has various standard data types that are used to define the operations possible on them and the storage method for each of them.



Numeric Data Types in Python

Integers – This value is represented by int class.

Float – This value is represented by the float class.

Complex Numbers – A complex number is represented by a complex class. **For example** – 2+3j

Int:

Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

```
>>> print(24656354687654+2)
24656354687656
>>> print(20)
20
>>> print(0b10)
2
>>> print(0B10)
2
>>> print(0X20)
32
>>> 20
20
>>> 0b10
2
>>> a=10
>>> print(a)
10
```

To verify the type of any object in Python, use the type() function:

```
>>> type(10)
<class 'int'>
>>> a=11
>>> print(type(a)) <class 'int'>
```

Float:

Float, or "floating point number" is a number, positive or negative, containing one or more decimals. Float can also be scientific numbers with an "e" to indicate the power of 10.

```
>>> y=2.8
>>> y
2.8
>>> y=2.8
>>> print(type(y))
<class 'float'>
>>> type(.4)
<class 'float'>
>>> 2.
2.0
```

Example:

```
x = 35e3
y = 12E4
z = -87.7e100
print(type(x))
print(type(y))
print(type(z))
```

Output:

```
<class 'float'>
<class 'float'>
<class 'float'>
```

Boolean:

Objects of Boolean type may have one of two values, True or False:

```
>>> type(True)
<class 'bool'>
>>> type(False)
<class 'bool'>
```

String:

Strings in Python are identified as a contiguous set of characters represented in the quotation marks. Python allows for either pairs of single or double quotes.

- ❖ 'hello' is the same as "hello".
- ❖ Strings can be output to screen using the print function. For example: print("hello").

```
>>> print("MIT college")
```

```
MIT college
>>> type("MIT college")
<class 'str'>
>>> print('MIT college') MIT college
>>> " "
''
```

If you want to include either type of quote character within the string, the simplest way is to delimit the string with the other type. If a string is to contain a single quote, delimit it with double quotes and vice versa:

```
>>> print("MIT is an autonomous (') college")
MIT is an autonomous (') college
>>> print('MIT is an autonomous (") college')
MIT is an autonomous (") college
```

Suppressing Special Character:

Specifying a backslash (\) in front of the quote character in a string “escapes” it and causes Python to suppress its usual special meaning. It is then interpreted simply as a literal single quote character:

```
>>> print("MIT is an autonomous (\') college")
MIT is an autonomous (') college
>>> print('MIT is an autonomous (\") college')
MIT is an autonomous (") college
```

The following is a table of escape sequences which cause Python to suppress the usual special interpretation of a character in a string:

```
>>> print('a\\...b')
a...b
>>> print('a\\b\\c')
abc
>>> print('a \\n b')
a b
>>> print("MIT \\n college") MIT
college
```

In Python (and almost all other common computer languages), a tab character can be specified by the escape sequence \t:

```
>>> print("a\\tb")
a      b
```

Escape Sequence	Usual Interpretation of Character(s) After Backslash	“Escaped” Interpretation
\'	Terminates string with single quote opening delimiter	Literal single quote (') character
\"	Terminates string with double quote opening delimiter	Literal double quote (") character
\newline	Terminates input line	Newline is ignored
\\	Introduces escape sequence	Literal backslash (\) character

Indentation:

- **Definition:** Whitespace at the beginning of a line that indicates a block of code.
- **Importance:** Indentation is crucial in Python; it defines the scope of loops, functions, and conditional statements.

Example:

```
if x > 0:
    print("Positive")
```

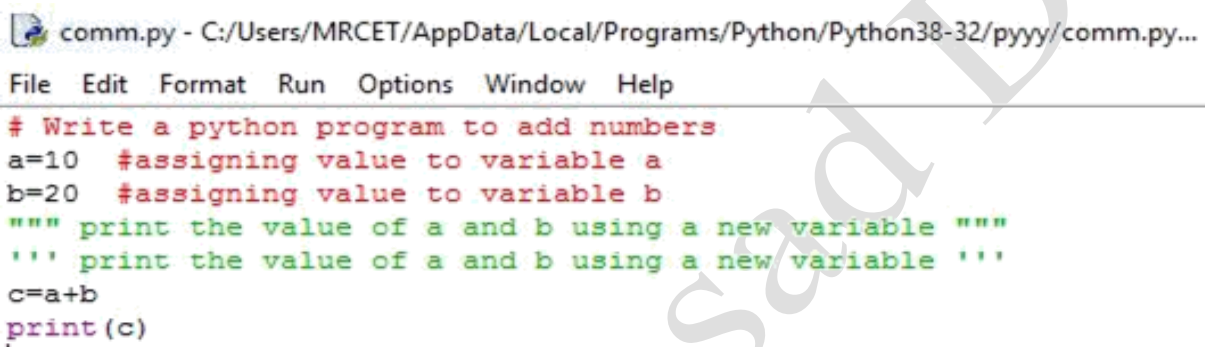
Comments:

- **Single-line Comments:** Use # to comment a single line.

Example: # This is a comment

- **Multi-line Comments:** Use triple quotes (""" or """).

Example:



The screenshot shows a Python IDE window titled 'comm.py - C:/Users/MRCET/AppData/Local/Programs/Python/Python38-32/pyyy/comm.py...'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code editor contains the following text:

```
# Write a python program to add numbers
a=10 #assigning value to variable a
b=20 #assigning value to variable b
""" print the value of a and b using a new variable """
''' print the value of a and b using a new variable '''
c=a+b
print(c)
```

Built-in Functions:

- **Console Input:** Use input() to take user input.

Example:

```
name = input("Enter your name: ")
```

- **Console Output:** Use print() to display output.

Example:

```
print("Hello, " + name)
```

Type Conversions:

Convert between types using built-in functions:

- **int():** Converts to integer.
- **float():** Converts to float.
- **str():** Converts to string.

Example:

```
x = "10"
y = int(x) + 5 # y is now 15
```

Python Libraries:

- **Definition:** Collections of pre-written code that you can use to perform common tasks.
- **Examples:** math, random, datetime.

Importing Libraries with Examples:

- **Importing a library:** Use the import statement.

Example:

```
import math
print(math.sqrt(16)) # Output: 4.0
```

- **Importing specific functions:** Use from keyword.

Example:

```
from random import randint
print(randint(1, 10)) # Outputs a random integer between 1 and 10
```

- **Importing with an alias:** Use as keyword.

Example:

```
import numpy as np
array = np.array([1, 2, 3])
```

Python Control Flow

Types of Control Flow:

Python supports the following types of control flow:

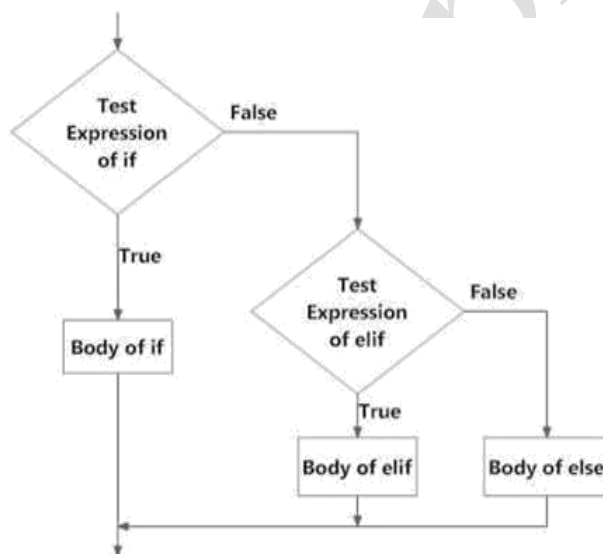
1. **Sequential Flow:** Statements are executed in the order they appear in the code.
2. **Conditional Flow:** Execution of code depends on certain conditions.
3. **Iterative Flow:** Execution of a block of code is repeated multiple times.

Control Flow Statements

1. if-else Statements

- ❖ **if statement:** Executes a block of code if a condition is true.
- ❖ **else statement:** Executes a block of code if the if condition is false.
- ❖ **elif statement:** Allows for multiple conditions to be checked.

if – elif – else Statement Flowchart:



Example:

```
x = 10
if x > 0:
    print("Positive")
elif x < 0:
    print("Negative")
else:
```

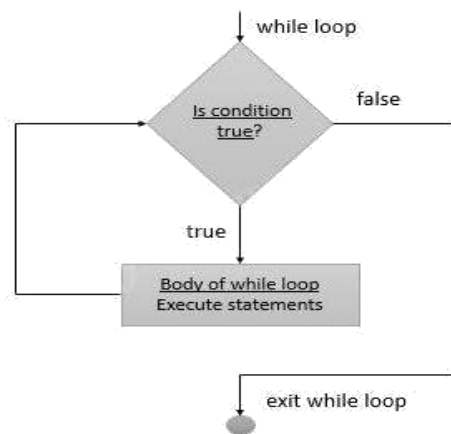
```
print("Zero")
```

2. while Loop

- ❖ Executes a block of code as long as a condition is true.
- ❖ Can be used to implement various types of loops, such as counting loops, infinite loops, and nested loops.
- ❖ Loops are either infinite or conditional. Python while loop keeps reiterating a block of code defined inside it until the desired condition is met.
- ❖ The while loop contains a boolean expression and the code inside the loop is repeatedly executed as long as the boolean expression is true.
- ❖ The statements that are executed inside while can be a single line of code or a block of multiple statements.
- ❖ **Syntax:**

```
while(expression):  
    Statement(s)
```

Flowchart:



Example:

```
count = 0  
while count < 5:  
    print(count)  
    count += 1
```

3. break Statement

- ❖ Terminates the current loop and transfers execution to the statement immediately following the loop.

Example:

```
while True:  
    user_input = input("Enter a number (or 'q' to quit): ")  
    if user_input == 'q':  
        break  
    print("You entered:", user_input)
```

4. continue Statement

- ❖ Skips the current iteration of the loop and moves to the next iteration.

Example:

```
for i in range(1, 11):  
    if i % 2 == 0:  
        continue  
    print(i)
```

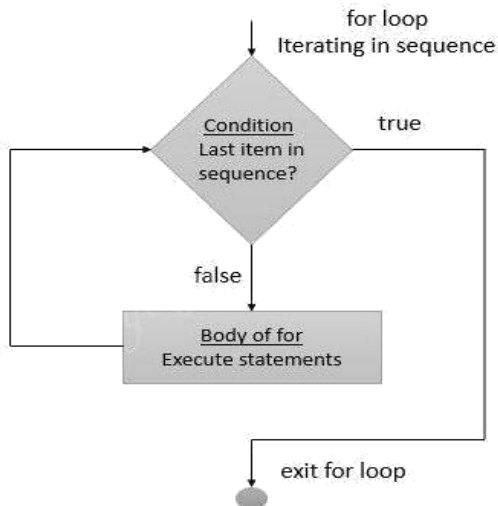
5. for Loop

- ❖ Iterates over the elements of a sequence (such as a list, tuple, or string).
- ❖ Can be used to implement various types of loops, such as counting loops, iterating over collections, and nested loops.

Example:

```
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Flowchart:



6. range() Function

- ❖ Generates a sequence of numbers.
- ❖ Can be used in for loops to iterate over a specific range of numbers.

The range () function has three types such as single parametric, two parametric and three parametric.

Example 1: for Single parametric

```
for i in range(5): # range(upto)
    print(i) # Output: 0, 1, 2, 3, 4
```

Example 2: for Two parametric

```
for i in range(1,5): # range(start,upto)
    print(i) # Output: 1, 2, 3, 4
```

Example 3: for Three parametric

```
for i in range(1,5,2): # range(start,upto,step)
    print(i) # Output: 1, 3
```

7. exit() Function

- ❖ Terminates the Python program and returns control to the operating system.

Example:

```
if user_input == 'q': # if user input is q then terminate the execution.
    exit()
```

Exception Handling in Python

Types of Errors

In Python, there are two main types of errors:

1. **Syntax Errors:** Occur when the Python parser detects an incorrect statement.

Example: `print("Hello, World!` (Missing closing parenthesis)

2. **Exceptions:** Runtime errors that occur during the execution of a program.

Example: `print(10 / 0)` (ZeroDivisionError)

Exceptions

Python Errors and Built-in Exceptions: Python (interpreter) raises exceptions when it encounters errors. When writing a program, we, more often than not, will encounter errors. Error caused by not following the proper structure (syntax) of the language is called syntax error or parsing error

1. ZeroDivisionError:

ZeroDivisionError in Python indicates that the second argument used in a division (or modulo) operation was zero.

2. OverflowError:

OverflowError in Python indicates that an arithmetic operation has exceeded the limits of the current Python runtime. This is typically due to excessively large float values, as integer values that are too big will opt to raise memory errors instead.

3. ImportError:

It is raised when you try to import a module which does not exist. This may happen if you made a typing mistake in the module name or the module doesn't exist in its standard path. In the example below, a module named "non_existing_module" is being imported but it doesn't exist, hence an import error exception is raised.

4. IndexError:

An IndexError exception is raised when you refer a sequence which is out of range. In the example below, the list abc contains only 3 entries, but the 4th index is being accessed, which will result an IndexError exception.

5. TypeError:

When two unrelated type of objects are combined, TypeError exception is raised. In example below, an int and a string is added, which will result in TypeError exception.

6. IndentationError:

Unexpected indent. As mentioned in the "expected an indented block" section, Python not only insists on indentation, it insists on consistent indentation. You are free to choose the number of spaces of indentation to use, but you then need to stick with it.

7. Syntax errors:

These are the most basic type of error. They arise when the Python parser is unable to understand a line of code. Syntax errors are almost always fatal, i.e. there is almost never a way to successfully execute a piece of code containing syntax errors.

8. Run-time error:

A run-time error happens when Python understands

What you are saying, but runs into trouble when following your instructions.

9. Key Error:

Python raises a `KeyError` whenever a `dict()` object is requested (using the format `a = adict[key]`) and the key is not in the dictionary.

10. Value Error:

In Python, a value is the information that is stored within a certain object. To encounter a `ValueError` in Python means that is a problem with the content of the object you tried to assign the value to.

Python has many built-in exceptions which forces your program to output an error when something in it goes wrong. In Python, users can define such exceptions by creating a new class. This exception class has to be derived, either directly or indirectly, from `Exception` class. You can also define and raise your own custom exceptions.

Exception Handling

Python provides a way to handle exceptions using the `try-except` statement. This allows you to anticipate and manage exceptions that may occur during the execution of your code.

1. try-except Statement

The basic structure of the `try-except` statement is as follows:

```
try:
    # code that might raise an exception
    pass
except ExceptionType:
    # code to handle the exception
    pass
```

You can have multiple `except` blocks to handle different types of exceptions:

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: Division by zero")
except TypeError:
    print("Error: Incompatible types")
```

2. finally Clause

The `finally` clause is used to specify a block of code that will be executed regardless of whether an exception is raised or not. This is useful for cleaning up resources, such as closing files or database connections.

```
try:
    file = open("file.txt", "r")
    content = file.read()
    print(content)
except FileNotFoundError:
    print("Error: File not found")
finally:
    file.close()
```

3. Raising Exceptions

You can also raise your own exceptions using the `raise` statement:

```
def divide(a, b):
    if b == 0:
        raise ZeroDivisionError("Error: Division by zero")
    return a / b

try:
    result = divide(10, 0)
except ZeroDivisionError as e:
    print(e)
```

Python Functions

Types of Functions

In Python, there are two main types of functions:

1. **Built-in Functions:** Functions that are part of the Python standard library, such as `print()`, `len()`, `range()`, `abs()`, `all()`, `ascii()`, `bool()`, etc.

Example:

```
integer = -20
print('Absolute value of -20 is:', abs(integer))
```

Output:

Absolute value of -20 is: 20

2. **User-Defined Functions:** Functions that you create yourself to perform specific tasks.

Function Definition

Function definition consists of following components:

1. Keyword **def** indicates the start of function header.
2. A function name to uniquely identify it. Function naming follows the same rules of writing identifiers in Python.
3. Parameters (arguments) through which we pass values to a function. They are optional.
4. A **colon (:)** to mark the end of function header.
5. Optional documentation string (docstring) to describe what the function does.
6. One or more valid python statements that make up the function body. Statements must have same indentation level (usually 4 spaces).
7. An optional return statement to return a value from the function.

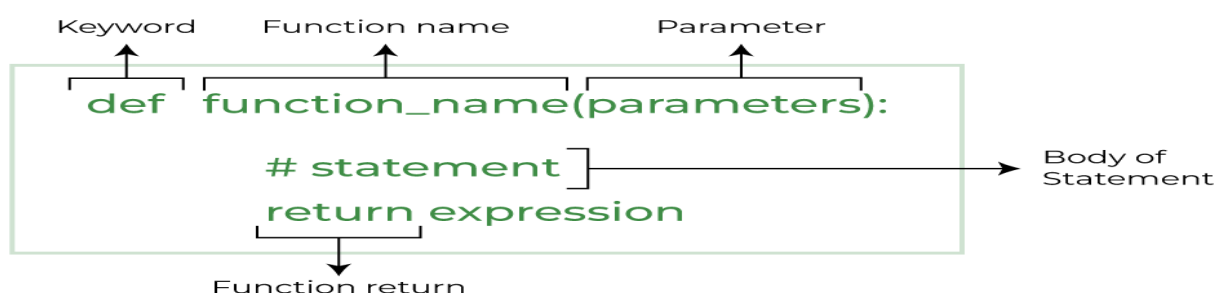
The syntax for defining a function in Python is as follows:

```
def function_name(parameters):
```

```
    """Docstring (optional)"""
```

```
    # Function body
```

```
    return value
```



Function Calling

To call a function, you simply use the function name followed by parentheses:

function_name(arguments)

Example:

```
def add(x,y):  
    c=x+y  
    return c
```

```
print(add(5,4))
```

Output:

9

Flow of Execution:

8. The order in which statements are executed is called the flow of execution
9. Execution always begins at the first statement of the program.
10. Statements are executed one at a time, in order, from top to bottom.
11. Function definitions do not alter the flow of execution of the program, but remember that statements inside the function are not executed until the function is called.
12. Function calls are like a bypass in the flow of execution. Instead of going to the next statement, the flow jumps to the first line of the called function, executes all the statements there, and then comes back to pick up where it left off.

Note: When you read a program, don't read from top to bottom. Instead, follow the flow of execution. This means that you will read the **def** statements as you are scanning from top to bottom, but you should skip the statements of the function definition until you reach a point where that function is called.

Example:

```
#example for flow of execution  
print("welcome")  
for x in range(3):  
    print(x)  
print("Good morning college")
```

Output:

welcome

0

1

2

Good morning college

Benefits of Using Functions

- **Reusability:** Write once, use multiple times.
- **Modularity:** Break complex tasks into smaller parts.
- **Readability:** Makes code more organized and easier to understand.
- **Maintainability:** Easier to update and debug.

Passing Parameters/Arguments

Functions can accept parameters, which are variables that are passed into the function when it is called. These parameters are used within the function body.

```
def greet(name):  
    print(f"Hello, {name}!")  
greet("Alice") # Output: Hello, Alice!
```

The return Statement

The return statement is used to exit a function and go back to the place from where it was called. This statement can contain expression which gets evaluated and the value is returned. If there is no expression in the statement or the return statement itself is not present inside a function, then the function will return the None object.

```
def add(a, b):  
    return a + b  
result = add(5, 3)  
print(result) # Output: 8
```

Default Parameters

Functions can have default parameter values, which are used when the parameter is not provided during the function call.

```
def greet(name="World"):  
    print(f"Hello, {name}!")  
greet() # Output: Hello, World!  
greet("Alice") # Output: Hello, Alice!
```

Command Line Arguments

Python's sys.argv list allows you to pass arguments to your script from the command line.

```
import sys  
if len(sys.argv) > 1:  
    name = sys.argv[1]  
else:  
    name = "World"  
print(f"Hello, {name}!")
```

Keyword Arguments

Keyword arguments allow you to specify the parameter name when calling a function, making the code more readable and flexible. When we call a function with some values, these values get assigned to the arguments according to their position. Python allows functions to be called using keyword arguments. When we call functions in this way, the order (position) of the arguments can be changed.

(Or)

If you have some functions with many parameters and you want to specify only some of them, then you can give values for such parameters by naming them - this is called keyword arguments - we use the name (keyword) instead of the position (which we have been using all along) to specify the arguments to the function. There are two advantages - one, using the function is easier since we do not need to worry about the order of the arguments. Two, we can give values to only those parameters which we want, provided that the other parameters have default argument values.

```
def calculate_area(length, width):  
    return length * width  
  
# Calling the function with keyword arguments  
area = calculate_area(length=5, width=3)  
print(area) # Output: 15
```

Recursive Functions

A recursive function is a function that calls itself to solve a problem. This can be a powerful technique for certain types of problems.

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n-1)  
print(factorial(5)) # Output: 120
```

Scope and Lifetime of Variables

Variables in Python have a defined scope and lifetime:

- **Local Scope:** Variables defined within a function are local and can only be accessed within that function.
- **Global Scope:** Variables defined outside of any function are global and can be accessed from anywhere in the code.
- **Lifetime:** Variables exist only for the duration of the function call and are destroyed when the function returns.

```
x = 10 # Global variable  
def my_function():  
    y = 5 # Local variable  
    print(x) # Can access global variable  
    print(y) # Can access local variable  
my_function() # Output: 10, 5  
print(y) # Error: y is not defined
```

Unit – 2

Strings: Creating and Storing Strings; Accessing String Characters; the str() function; Operations on Strings- Concatenation, Comparison, Slicing and Joining, Traversing; Format Specifiers; Escape Sequences; Raw and Unicode Strings; Python String Methods.

Lists: Creating Lists; Operations on Lists; Built-in Functions on Lists; Implementation of Stacks and Queues using Lists; Nested Lists.

Dictionaries: Creating Dictionaries; Operations on Dictionaries; Built-in Functions on Dictionaries; Dictionary Methods; Populating and Traversing Dictionaries.

Tuples and Sets: Creating Tuples; Operations on Tuples; Built-in Functions on Tuples; Tuple Methods; Creating Sets; Operations on Sets; Built-in Functions on Sets; Set Methods

File Handling: File Types; Operations on Files– Create, Open, Read, Write, Close Files; File Names and Paths; Format Operator.

Python Strings

Creating and Storing Strings

A String in Python consists of a series or sequence of characters - letters, numbers, and special characters. Strings in Python are defined by enclosing characters in single quotes ('), double quotes ("), or triple quotes (''' or ''').

```
# Creating strings
my_string = "Hello, World!"
another_string = 'Python is awesome'
multiline_string = """This is a
multiline
string."""
```

Accessing String Characters

Individual characters in a string can be accessed using indexing, where the first character has an index of 0.

```
my_string = "Python"
print(my_string[0]) # Output: 'P'
print(my_string[3]) # Output: 'h'
```

The str() Function

The str() function is used to convert other data types, such as integers and floats, to strings.

```
age = 25
age_string = str(age)
print(age_string) # Output: '25'
```

String Operations

1. Concatenation

Strings can be concatenated using the + operator.

```
first_name = "John"
last_name = "Doe"
full_name = first_name + " " + last_name
print(full_name) # Output: "John Doe"
```

2. Comparison

Strings can be compared using comparison operators like <, >, ==, etc.

```
print("apple" < "banana") # Output: True
print("Python" == "python") # Output: False
```

3. Slicing and Joining

Strings can be sliced to extract substrings, and the join() function can be used to concatenate a list of strings.

```
message = "Hello, World!"
print(message[0:5]) # Output: "Hello"
print(" ".join(["Python", "is", "awesome"])) # Output: "Python is awesome"
```

4. Traversing

Strings can be traversed using a for loop to access each character.

```
for char in "Python":
    print(char)
```

Format Specifiers

Format specifiers allow you to insert values into strings using placeholders.

```
name = "Alice"
age = 25
print("My name is {} and I'm {} years old.".format(name, age))
print(f"My name is {name} and I'm {age} years old.") # f-strings (Python 3.6+)
```

Escape Sequences

Escape sequences allow you to include special characters in strings.

```
print("This is a \"quoted\" string.")
print("This is a\nmultiline\nstring.")
```

Raw and Unicode Strings

Raw strings (r"...") preserve backslashes, and Unicode strings (u"...") allow the use of Unicode characters.

```
print(r"C:\Users\username\Documents")
print(u"Hello, 世界!")
```

String Methods

Python provides a wide range of string methods for manipulating and processing strings.

```
my_string = " Python is awesome "
print(my_string.strip()) # Output: "Python is awesome"
print(my_string.upper()) # Output: " PYTHON IS AWESOME "
print(my_string.replace("Python", "Java")) # Output: " Java is awesome "
```

String functions and methods:

There are many methods to operate on String.

S.no	Method name	Description
1.	isalnum()	Returns true if string has at least 1 character and all characters are alphanumeric and false otherwise.
2.	isalpha()	Returns true if string has at least 1 character and all characters are

		alphabetic and false otherwise.
3.	isdigit()	Returns true if string contains only digits and false otherwise.
4.	islower()	Returns true if string has at least 1 cased character and all cased characters are in lowercase and false otherwise.
5.	isnumeric()	Returns true if a string contains only numeric characters and false otherwise.
6.	isspace()	Returns true if string contains only whitespace characters and false otherwise.
7.	istitle()	Returns true if string is properly "titlecased" and false otherwise.
8.	isupper()	Returns true if string has at least one cased character and all cased characters are in uppercase and false otherwise.
9.	replace(old, new [, max])	Replaces all occurrences of old in string with new or at most max occurrences if max given.
10.	split()	Splits string according to delimiter str (space if not provided) and returns list of substrings;
11.	count()	Occurrence of a string in another string
12.	find()	Finding the index of the first occurrence of a string in another string
13.	swapcase()	Converts lowercase letters in a string to uppercase and viceversa
14.	startswith(str, beg=0,end=len(string))	Determines if string or a substring of string (if starting index beg and ending index end are given) starts with substring str; returns true if so and false otherwise.

Note:

All the string methods will be returning either true or false as the result

1. isalnum():

isalnum() method returns true if string has at least 1 character and all characters are alphanumeric and false otherwise.

Example:

```
>>> string="123alpha"
>>> string.isalnum()
True
```

2. isalpha():

isalpha() method returns true if string has at least 1 character and all characters are alphabetic and false otherwise.

Example:

```
>>> string="nikhil"
>>> string.isalpha()
True
```

3. isdigit():

isdigit() returns true if string contains only digits and false otherwise.

Example:

```
>>> string="123456789"
>>> string.isdigit()
True
```

4. islower():

islower() returns true if string has characters that are in lowercase and false otherwise.

Example:

```
>>> string="nikhil"  
>>> string.islower()  
True
```

5. isnumeric():

isnumeric() method returns true if a string contains only numeric characters and false otherwise.

Example:

```
>>> string="123456789"  
>>> string.isnumeric()  
True
```

6. isspace():

isspace() returns true if string contains only whitespace characters and false otherwise.

Example:

```
>>> string=" "  
>>> string.isspace()  
True
```

7. istitle()

istitle() method returns true if string is properly "titlecased"(starting letter of each word is capital) and false otherwise

Example:

```
>>> string="Nikhil Is Learning"  
>>> string.istitle()  
True
```

8. isupper()

isupper() returns true if string has characters that are in uppercase and false otherwise.

Example:

```
>>> string="HELLO"  
>>> string.isupper()  
True
```

9. replace()

replace() method replaces all occurrences of old in string with new or at most max occurrences if max given.

Example:

```
>>> string="Nikhil Is Learning"  
>>> string.replace('Nikhil','Neha') 'Neha Is Learning'
```

10. split()

split() method splits the string according to delimiter str (space if not provided)

Example:

```
>>> string="Nikhil Is Learning"  
>>> string.split()  
['Nikhil', 'Is', 'Learning']
```

11. count()

count() method counts the occurrence of a string in another string

Example:

```
>>> string='Nikhil Is Learning'  
>>> string.count('i')
```

12. find()

Find() method is used for finding the index of the first occurrence of a string in another string

Example:

```
>>> string="Nikhil Is Learning"
>>> string.find('k')
2
```

13. swapcase()

converts lowercase letters in a string to uppercase and viceversa

Example:

```
>>> string="HELLO"
>>> string.swapcase()
'hello'
```

14. startswith()

Determines if string or a substring of string (if starting index beg and ending index end are given) starts with substring str; returns true if so and false otherwise.

Example:

```
>>> string="Nikhil Is Learning"
>>> string.startswith('N')
True
```

15. endswith()

Determines if string or a substring of string (if starting index beg and ending index end are given) ends with substring str; returns true if so and false otherwise.

Example:

```
>>> string="Nikhil Is Learning"
>>> string.endswith('g')
True
```

Lists in Python

Creating Lists

Lists in Python are defined using square brackets [] and can contain elements of different data types. It is a general purpose most widely used in data structures. List is a collection which is ordered and changeable and allows duplicate members. (Grow and shrink as needed, sequence type, sortable).

To use a list, you must declare it first. Do this using square brackets and separate values with commas. We can construct / create list in many ways.

```
my_list = [1, 2, 3, "four", 5.0]
```

List Operations

Lists support various operations, such as indexing, slicing, concatenation, and repetition.

```
print(my_list[0]) # Output: 1
print(my_list[1:4]) # Output: [2, 3, 'four']
print(my_list + [6, 7]) # Output: [1, 2, 3, 'four', 5.0, 6, 7]
print(my_list * 2) # Output: [1, 2, 3, 'four', 5.0, 1, 2, 3, 'four', 5.0]
```

Built-in List Functions

Python provides numerous built-in functions for working with lists, such as len(), min(), max(), sum(), and sorted().

```
print(len(my_list)) # Output: 5
print(min(my_list)) # Output: 1
```

```
print(max(my_list)) # Output: 'four'
print(sum(my_list)) # Output: 11.0
print(sorted(my_list)) # Output: [1, 2, 3, 5.0, 'four']
```

Method	Description
append()	Adds an element at the end of the list
clear()	Removes all the elements from the list
copy()	Returns a copy of the list
count()	Returns the number of elements with the specified value
extend()	Add the elements of a list (or any iterable), to the end of the current list
index()	Returns the index of the first element with the specified value
insert()	Adds an element at the specified position
pop()	Removes the element at the specified position
remove()	Removes the first item with the specified value
reverse()	Reverses the order of the list
sort()	Sorts the list

Note: Python does not have built-in support for Arrays, but Python Lists can be used instead.

Example:

```
>>> college=["MIT","it","cse"]
>>> college.append("autonomous")
>>> college
['MIT', 'it', 'cse', 'autonomous']

>>> college.append("eee")
>>> college.append("ece")
>>> college
['MIT', 'it', 'cse', 'autonomous', 'eee', 'ece']

>>> college.pop()
'ece'
>>> college
['MIT', 'it', 'cse', 'autonomous', 'eee']

>>> college.pop(4)
'eee'
>>> college
['MIT', 'it', 'cse', 'autonomous']

>>> college.remove("it")
>>> college
['MIT', 'cse', 'autonomous']
```

Stacks and Queues using Lists

Lists can be used to implement basic data structures like stacks and queues.

```
# Stack
stack = [1, 2, 3]
stack.append(4) # Push
print(stack.pop()) # Output: 4 (Pop)

# Queue
queue = [1, 2, 3]
queue.append(4) # Enqueue
print(queue.pop(0)) # Output: 1 (Dequeue)
```

Nested Lists

Lists can contain other lists as elements, creating nested data structures.

```
nested_list = [[1, 2], [3, 4], [5, 6]]
print(nested_list[1][0]) # Output: 3
```

List methods:

The list data type has some more methods. Here are all of the methods of list objects: del(), append(), extend(), insert(), pop(), remove(), reverse(), and sort()

delete: Delete a list or an item from a list

```
>>> x=[5,3,8,6]
>>> del(x[1]) #deletes the index position 1 in a list
>>> x
[5, 8, 6]
>>> del(x)
>>> x # complete list gets deleted
```

append: Append an item to a list

```
>>> x=[1,5,8,4]
>>> x.append(10)
>>> x
[1, 5, 8, 4, 10]
```

extend: Append a sequence to a list.

```
>>> x=[1,2,3,4]
>>> y=[3,6,9,1]
>>> x.extend(y)
>>> x
[1, 2, 3, 4, 3, 6, 9, 1]
```

insert: To add an item at the specified index, use the insert () method:

```
>>> x=[1,2,4,6,7]
>>> x.insert(2,10) #insert(index no, item to be inserted)
>>> x
[1, 2, 10, 4, 6, 7]

>>> x.insert(4,['a',11])
>>> x
[1, 2, 10, 4, ['a', 11], 6, 7]
```

pop: The pop() method removes the specified index, (or the last item if index is not specified) or simply pops the last item of list and returns the item.

```
>>> x=[1, 2, 10, 4, 6, 7]
>>> x.pop()
7
>>> x
[1, 2, 10, 4, 6]
>>> x=[1, 2, 10, 4, 6]
>>> x.pop(2)
10
>>> x
[1, 2, 4, 6]
```

remove: The remove() method removes the specified item from a given list.

```
>>> x=[1,33,2,10,4,6]
```

```
>>> x.remove(33)
>>> x
[1, 2, 10, 4, 6]
>>> x.remove(4)
>>> x
[1, 2, 10, 6]
```

reverse: Reverses the order of the list

```
>>> x=[1,2,3,4,5,6,7]
>>> x.reverse()
>>> x
[7, 6, 5, 4, 3, 2, 1]
```

sort: sort the given list in ascending order

```
>>> x=[7, 6, 5, 4, 3, 2, 1]
>>> x.sort()
>>> x
[1, 2, 3, 4, 5, 6, 7]
>>> x=[10,1,5,3,8,7]
>>> x.sort(reverse = True) # descending order
>>> x
[10, 8, 7, 5, 3, 1]
```

List loop:

Loops are control structures used to repeat a given section of code a certain number of times or until a particular condition is met.

Method #1: For loop

```
#list of items
list = ['M','R','C','E','T']
i = 1
#Iterating over the list
for item in list:
    print ('college ',i,' is ',item)
    i = i+1
```

Output:

```
college 1 is M
college 2 is R
college 3 is C
college 4 is E
college 5 is T
```

Method #2: For loop and range()

In case we want to use the traditional for loop which iterates from number x to number y.

```
# Python3 code to iterate over a list
list = [1, 3, 5, 7, 9]
# getting length of list
length = len(list)
# Iterating the index
# same as 'for i in range(len(list))'
for i in range(length):
    print(list[i])
```

Output:

```
1
```

3
5
7
9

Method #3: using while loop

```
# Python3 code to iterate over a list
list = [1, 3, 5, 7, 9]
# Getting length of list
length = len(list)

i = 0
# Iterating using while loop
while i < length:
    print(list[i])
    i += 1
```

Mutability:

The list is mutable because it can be changed after it is created.

Cloning Lists:

If we want to modify a list and also keep a copy of the original, we need to be able to make a copy of the list itself, not just the reference. This process is sometimes called cloning, to avoid the ambiguity of the word copy. The easiest way to clone a list is to use the slice operator. Taking any slice of a creates a new list. In this case the slice happens to consist of the whole list.

Example:

```
a = [81, 82, 83]
b = a[:]# make a clone using slice
print(a == b)
print(a is b)
b[0] = 5
print(a)
print(b)
```

Output:

```
True
False
[81, 82, 83]
[5, 82, 83]
Now we are free to make changes to b without worrying about a
```

List parameters:

Passing a list as an argument actually passes a reference to the list, not a copy of the list. Since lists are mutable, changes made to the elements referenced by the parameter change the same list that the argument is referencing.

#for example, the function below takes a list as an argument and multiplies each element in the list by 2:

```
def doubleStuff(List):
    """ Overwrite each element in aList with double its value. """
    for position in range(len(List)):
        List[position] = 2 * List[position]
things = [2, 5, 9]
print(things)
doubleStuff(things)
```

```
print(things)
```

Output:

```
[2, 5, 9]
```

```
[4, 10, 18]
```

List comprehension:

List comprehensions provide a concise way to create lists. Common applications are to make new lists where each element is the result of some operations applied to each member of another sequence or iterable, or to create a subsequence of those elements that satisfy a certain condition.

For example, assume we want to create a list of squares, like:

```
>>> list1=[]
```

```
>>> for x in range(10): list1.append(x**2)
```

```
>>> list1
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

(or)

This is also equivalent to

```
>>> list1=list(map(lambda x:x**2, range(10)))
```

```
>>> list1
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

(or)

Which is more concise and readable.

```
>>> list1=[x**2 for x in range(10)]
```

```
>>> list1
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Similarly some examples:

```
>>> x=[m for m in range(8)]
```

```
>>> print(x)
```

```
[0, 1, 2, 3, 4, 5, 6, 7]
```

```
>>> x=[z**2 for z in range(10) if z>4]
```

```
>>> print(x)
```

```
[25, 36, 49, 64, 81]
```

```
>>> x=[x ** 2 for x in range (1, 11) if x % 2 == 1]
```

```
>>> print(x)
```

```
[1, 9, 25, 49, 81]
```

```
>>> a=5
```

```
>>> table = [[a, b, a * b] for b in range(1, 11)]
```

```
>>> for i in table: print(i)
```

```
[5, 1, 5]
```

```
[5, 2, 10]
```

```
[5, 3, 15]
```

```
[5, 4, 20]
```

```
[5, 5, 25]
```

```
[5, 6, 30]
```

```
[5, 7, 35]
```

```
[5, 8, 40]
```

```
[5, 9, 45]
```

```
[5, 10, 50]
```

Tuples:

A tuple is a collection which is ordered and unchangeable. In Python tuples are written with round brackets. Supports all operations for sequences. Immutable, but member objects may be mutable. If the contents of a list shouldn't change, use a tuple to prevent items from accidentally being added, changed, or deleted. Tuples are more efficient than list due to python's implementation.

We can construct tuple in many ways:

```
X=() #no item tuple
X=(1,2,3)
X=tuple(list1)
X=1,2,3,4
```

Example:

```
>>> x=(1,2,3)
>>> print(x)
(1, 2, 3)
>>> x
(1, 2, 3)

>>> x=()
>>> x
()

>>> x=[4,5,66,9]
>>> y=tuple(x)
>>> y
(4, 5, 66, 9)

>>> x=1,2,3,4
>>> x
(1, 2, 3, 4)
```

Some of the operations of tuple are:

Access tuple items, Change tuple items, Loop through a tuple, count(), index(), length().

Access tuple items: Access tuple items by referring to the index number, inside square brackets

```
>>> x=('a','b','c','g')
>>> print(x[2])
c
```

Change tuple items: Once a tuple is created, you cannot change its values. Tuples are unchangeable.

```
>>> x=(2,5,7,'4',8)
>>> x[1]=10
Traceback (most recent call last):
  File "<pyshell#41>", line 1, in <module>

x[1]=10
TypeError: 'tuple' object does not support item assignment
>>> x
(2, 5, 7, '4', 8) # the value is still the same
```

Loop through a tuple: We can loop the values of tuple using for loop

```
>>> x=4,5,6,7,2,'aa'
>>> for i in x: print(i)
4
5
```

```
6
7
2
aa
```

count (): Returns the number of times a specified value occurs in a tuple

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
>>> x.count(2)
4
```

Index (): Searches the tuple for a specified value and returns the position of where it was found

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
>>> x.index(2)
1
```

(Or)

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
>>> y=x.index(2)
>>> print(y)
1
```

length (): To know the number of items or values present in a tuple, we use len().

```
>>> x=(1,2,3,4,5,6,2,10,2,11,12,2)
>>> y=len(x)
>>> print(y)
12
```

Tuple Assignment

Python has tuple assignment feature which enables you to assign more than one variable at a time. In here, we have assigned tuple 1 with the college information like college name, year, etc. and another tuple 2 with the values in it like number (1, 2, 3... 7).

Example:

```
>>> tup1 = ('MIT', 'eng college','2004','cse', 'it','csit');
>>> tup2 = (1,2,3,4,5,6,7);
>>> print(tup1[0])
MIT
>>> print(tup2[1:4])
(2, 3, 4)
```

Tuple 1 includes list of information of MIT

Tuple 2 includes list of numbers in it

We call the value for [0] in tuple and for tuple 2 we call the value between 1 and 4

Run the above code- It gives name MIT for first tuple while for second tuple it gives number (2, 3, 4)

Tuple as return values:

A Tuple is a comma separated sequence of items. It is created with or without (). Tuples are immutable.

```
# A Python program to return multiple values from a method using tuple
# This function returns a tuple
def fun():
    str = "MIT college"
    x = 20
    return str, x; # Return tuple, we could also
# write (str, x)
# Driver code to test above method str, x = fun() # Assign returned tuple
print(str)
print(x)
```

Output:
MIT college
20

Functions can return tuples as return values.

```
def circleInfo(r):  
    """ Return (circumference, area) of a circle of radius r """  
    c = 2 * 3.14159 * r  
    a = 3.14159 * r * r  
    return (c, a)  
  
print(circleInfo(10))
```

Output:
(62.8318, 314.159)

```
def f(x):  
    y0 = x + 1  
    y1 = x * 3  
    y2 = y0 ** y3  
    return (y0, y1, y2)
```

Tuple comprehension:

Tuple Comprehensions are special: The result of a tuple comprehension is special. You might expect it to produce a tuple, but what it does is produce a special "generator" object that we can iterate over.

Example:

```
>>> x = (i for i in 'abc') #tuple comprehension  
>>> x  
<generator object <genexpr> at 0x033EEC30>  
>>> print(x)  
<generator object <genexpr> at 0x033EEC30>
```

You might expect this to print as ('a', 'b', 'c') but it prints as <generator object <genexpr> at 0x02AAD710> The result of a tuple comprehension is not a tuple: it is actually a generator. The only thing that you need to know now about a generator now is that you can iterate over it, but ONLY ONCE.

So, given the code

```
>>> x = (i for i in 'abc')  
>>> for i in x: print(i)  
a  
b  
c
```

Create a list of 2-tuples like (number, square):

```
>>> z=[(x, x**2) for x in range(6)]  
>>> z  
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
```

Set:

Similarly to list comprehensions, set comprehensions are also supported:

```
>>> a = {x for x in 'abracadabra' if x not in 'abc'}  
>>> a  
{ 'r', 'd' }  
>>> x={3*x for x in range(10) if x>5}  
>>> x  
{24, 18, 27, 21}
```

Operations and Functions

Tuples and sets support various operations and built-in functions, such as indexing, membership testing, and set operations.

```
print(1 in my_set) # Output: True
print(len(my_tuple)) # Output: 3
print(set([1, 2, 3, 2, 1])) # Output: {1, 2, 3}
```

Dictionaries in Python

Creating Dictionaries

A dictionary is a collection which is unordered, changeable and indexed. In Python dictionaries are written with curly brackets {} and they have Key-value pairs.

```
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
X=dict([('a',3), ('b',4)])
X=dict('A'=1, 'B'=2)
```

Dictionary Operations

Dictionaries support operations like indexing, adding/modifying/deleting key-value pairs, and membership testing.

```
print(my_dict["name"]) # Output: 'Alice'
my_dict["email"] = "alice@example.com"
del my_dict["city"]
print("age" in my_dict) # Output: True
```

Output:

```
Alice
True
```

Built-in Dictionary Functions

Python provides built-in functions for working with dictionaries, such as len(), keys(), values(), and items().

```
print(len(my_dict)) # Output: 3
print(list(my_dict.keys())) # Output: ['name', 'age', 'email']
print(list(my_dict.values())) # Output: ['Alice', 25, 'alice@example.com']
print(list(my_dict.items())) # Output: [('name', 'Alice'), ('age', 25), ('email', 'alice@example.com')]
```

Output:

```
3
['name', 'age', 'email']
['Alice', 25, 'alice@example.com']
[('name', 'Alice'), ('age', 25), ('email', 'alice@example.com')]
```

Dictionary Methods

Dictionaries have various methods for manipulating and accessing their contents, such as get(), update(), and clear().

```
print(my_dict.get("city", "Unknown")) # Output: 'Unknown'
my_dict.update({"age": 26, "city": "San Francisco"})
my_dict.clear()
print(my_dict)
```

Output:

```
Unknown
{}
```

Method	Description
<code>clear()</code>	Remove all items form the dictionary.
<code>copy()</code>	Return a shallow copy of the dictionary.
<code>fromkeys(seq[, v])</code>	Return a new dictionary with keys from seq and value equal to v (defaults to None).
<code>get(key[,d])</code>	Return the value of key. If key doesnot exit, return d (defaults to None).
<code>items()</code>	Return a new view of the dictionary's items (key, value).
<code>keys()</code>	Return a new view of the dictionary's keys.
<code>pop(key[,d])</code>	Remove the item with key and return its value or d if key is not found. If d is not provided and key is not found, raises KeyError.
<code>popitem()</code>	Remove and return an arbitrary item (key, value). Raises KeyError if the dictionary is empty.
<code>setdefault(key[,d])</code>	If key is in the dictionary, return its value. If not, insert key with a value of d and return d (defaults to None).
<code>update([other])</code>	Update the dictionary with the key/value pairs from other, overwriting existing keys.
<code>values()</code>	Return a new view of the dictionary's values

Below are some dictionary operations:

To access specific value of a dictionary, we must pass its key,

```
>>> dict1 = {"brand": "MIT", "model": "college", "year": 2004}
>>> x = dict1["brand"]
>>> x
'MIT'
```

To access keys and values and items of dictionary:

```
>>> dict1 = {"brand": "MIT", "model": "college", "year": 2004}
>>> dict1.keys()
dict_keys(['brand', 'model', 'year'])
>>> dict1.values()
dict_values(['MIT', 'college', 2004])
>>> dict1.items()
dict_items([('brand', 'MIT'), ('model', 'college'), ('year', 2004)])
>>> for items in dict1.values(): print(items)
MIT
college
2004
>>> for items in dict1.keys(): print(items)
brand
model
year
>>> for i in dict1.items(): print(i)
```

```
('brand', 'MIT')
('model', 'college')
('year', 2004)
```

Some more operations like: Add/change, Remove, Length, Delete.

Add/change values: You can change the value of a specific item by referring to its key name

```
>>> dict1 = {"brand": "MIT", "model": "college", "year": 2004}
>>> dict1["year"] = 2005
>>> dict1
{'brand': 'MIT', 'model': 'college', 'year': 2005}
```

Remove(): It removes or pop the specific item of dictionary.

```
>>> dict1 = {"brand": "MIT", "model": "college", "year": 2004}
>>> print(dict1.pop("model"))
college
>>> dict1
{'brand': 'MIT', 'year': 2005}
```

Delete: Deletes a particular item.

```
>>> x = {1:1, 2:4, 3:9, 4:16, 5:25}
>>> del x[5]
>>> x
```

Length: we use len() method to get the length of dictionary.

```
>>> {1: 1, 2: 4, 3: 9, 4: 16}
{1: 1, 2: 4, 3: 9, 4: 16}
>>> y = len(x)
>>> y
4
```

Iterating over (key, value) pairs:

```
>>> x = {1:1, 2:4, 3:9, 4:16, 5:25}
>>> for key in x: print(key, x[key])
1      1
2      4
3      9
4      16
5      25
>>> for k,v in x.items(): print(k,v)
1      1
2      4
3      9
4      16
5      25
```

List of Dictionaries:

```
>>> customers = [{"uid":1,"name":"John"}, {"uid":2,"name":"Smith"},
{"uid":3,"name":"Andersson"}]
>>> print(customers)
[{'uid': 1, 'name': 'John'}, {'uid': 2, 'name': 'Smith'}, {'uid': 3, 'name': 'Andersson'}]
##      Print the uid and name of each customer
>>> for x in customers: print(x["uid"], x["name"])
1      John
2      Smith
3      Andersson
```

Modify an entry, This will change the name of customer 2 from Smith to Charlie

```
>>> customers[2]["name"]="charlie"
>>> print(customers)
[{'uid': 1, 'name': 'John'}, {'uid': 2, 'name': 'Smith'}, {'uid': 3, 'name': 'charlie'}]
```

Add a new field to each entry,

```
>>> for x in customers:
x["password"]="123456" # any initial value
>>> print(customers)
[{'uid': 1, 'name': 'John', 'password': '123456'}, {'uid': 2, 'name': 'Smith',
'password': '123456'}, {'uid': 3, 'name': 'charlie', 'password': '123456'}]
```

Delete a field,

```
>>> del customers[1]
>>> print(customers)
[{'uid': 1, 'name': 'John', 'password': '123456'}, {'uid': 3, 'name': 'charlie', 'password':
'123456'}]
>>> del customers[1]
>>> print(customers)
[{'uid': 1, 'name': 'John', 'password': '123456'}]
```

Delete all fields,

```
>>> for x in customers: del x["uid"]
>>> x
{'name': 'John', 'password': '123456'}
```

Comprehension:

Dictionary comprehensions can be used to create dictionaries from arbitrary key and value expressions:

```
>>> z={x: x**2 for x in (2,4,6)}
>>> z
{2: 4, 4: 16, 6: 36}
>>> dict11 = {x: x*x for x in range(6)}
>>> dict11
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

File Handling in Python

File:

A file is some information or data which stays in the computer storage devices. Python gives you easy ways to manipulate these files. Generally files divide in two categories, text file and binary file. Text files are simple text where as the binary files contain binary data which is only readable by computer.

File Types

Python can work with various file types, including text files, binary files, and CSV files.

Text files: In this type of file, Each line of text is terminated with a special character called EOL (End of Line), which is the new line character ('`\n`') in python by default.

We can create the text files by using the syntax:

Variable name=open ("file.txt", file mode)

Example:

```
f= open ("hello.txt","w+")
```

We declared the variable f to open a file named hello.txt. Open takes 2 arguments, the file that we want to open and a string that represents the kinds of permission or operation we want to do on the file. Here we used "w" letter in our argument, which indicates write and the plus sign that means it will create a file if it does not exist in library. The available option beside "w" are "r" for read and "a" for append and plus sign means if it is not there then create it. File Modes in Python:

Mode	Description	Behavior
r	Read-only mode.	Opens the file for reading. File must exist; otherwise, it raises an error.
rb	Read-only in binary mode.	Opens the file for reading binary data. File must exist; otherwise, it raises an error.
r+	Read and write mode.	Opens the file for both reading and writing. File must exist; otherwise, it raises an error.
rb+	Read and write in binary mode.	Opens the file for both reading and writing binary data. File must exist; otherwise, it raises an error.
w	Write mode.	Opens the file for writing. Creates a new file or truncates the existing file.
wb	Write in binary mode.	Opens the file for writing binary data. Creates a new file or truncates the existing file.
w+	Write and read mode.	Opens the file for both writing and reading. Creates a new file or truncates the existing file.
wb+	Write and read in binary mode.	Opens the file for both writing and reading binary data. Creates a new file or truncates the existing file.
a	Append mode.	Opens the file for appending data. Creates a new file if it doesn't exist.

Mode	Description	Behavior
ab	Append in binary mode.	Opens the file for appending binary data. Creates a new file if it doesn't exist.
a+	Append and read mode.	Opens the file for appending and reading. Creates a new file if it doesn't exist.
ab+	Append and read in binary mode.	Opens the file for appending and reading binary data. Creates a new file if it doesn't exist.
x	Exclusive creation mode.	Creates a new file. Raises an error if the file already exists.
xb	Exclusive creation in binary mode.	Creates a new binary file. Raises an error if the file already exists.
x+	Exclusive creation with read and write mode.	Creates a new file for reading and writing. Raises an error if the file exists.
xb+	Exclusive creation with read and write in binary mode.	Creates a new binary file for reading and writing. Raises an error if the file exists.

Binary files: In this type of file, there is no terminator for a line and the data is stored after converting it into machine understandable binary language.

File Operations

Python provides the `open()` function to interact with files, and the `with` statement to ensure proper file handling.

```
with open("example.txt", "w") as file:
    file.write("Hello, World!")
```

```
with open("example.txt", "r") as file:
    content = file.read()
    print(content) # Output: "Hello, World!"
```

File Names and Paths

Python's `os` module provides functions for working with file names and paths.

```
import os
```

```
print(os.path.join("documents", "file.txt")) # Output: 'documents/file.txt'
print(os.path.dirname("/home/user/documents/file.txt"))
# Output: '/home/user/documents'
```

Format Operator

The format operator `%` can be used to format strings, including file paths.

```
file_path = "/home/user/%s/file.txt" % "documents"
print(file_path) # Output: '/home/user/documents/file.txt'
```

Reading and Writing files:

The following image shows how to create and open a text file in notepad from command prompt

```
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

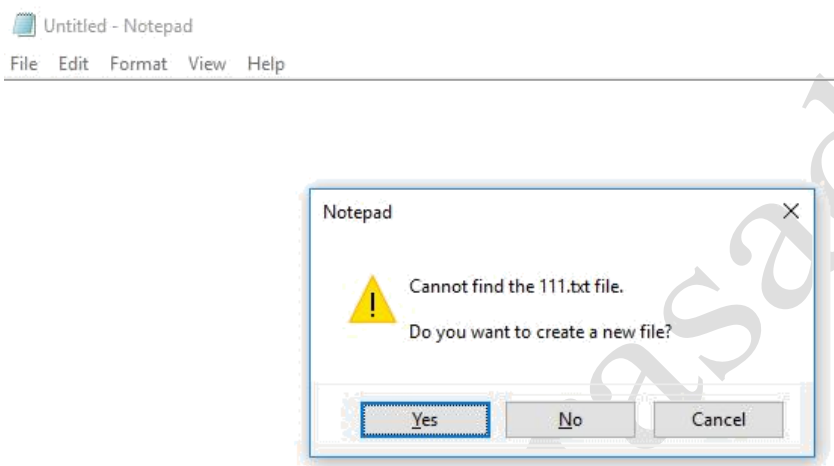
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>start notepad hello.txt

C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type hello.txt
Hello mrcet
good morning
how r u
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>
```

(or)

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>notepad 111.txt
```

Hit on enter then it shows the following whether to open or not?



Click on “yes” to open else “no” to cancel

Write a python program to open and read a file

```
a=open("one.txt","r")
print(a.read())
a.close()
```

Output:

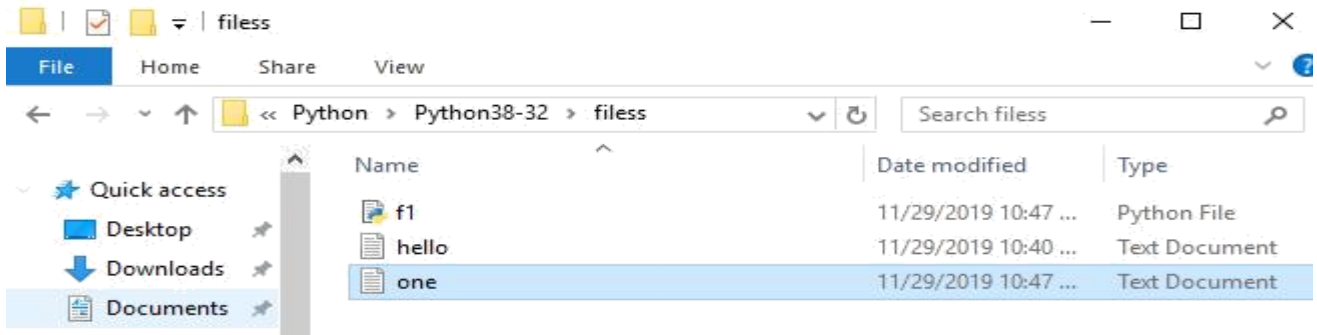
welcome to python programming

(or)

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>python f1.py
welcome to python programming
```

Note: All the program files and text files need to saved together in a particular file then only the program performs the operations in the given file mode

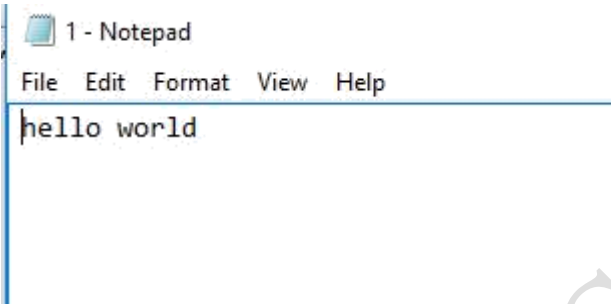
f.close() ---- This will close the instance of the file somefile.txt stored



Write a python program to open and write “hello world” into a file?

```
f=open("1.txt","a")
f.write("hello world")
f.close()
```

Output:



(or)

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type 1.txt
hello world
```

Note: In the above program the 1.txt file is created automatically and adds hello world into txt file.

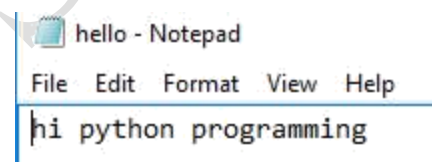
If we keep on executing the same program for more than one time then it append the data that many times

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type 1.txt
hello worldhello world
```

#Write a python program to write the content “hi python programming” for the existing file.

```
f=open("1.txt",'w')
f.write("hi python programming")
f.close()
```

Output:



In the above program the hello txt file consist of data like

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type hello.txt
Hello mrcet
good morning
how r u
```

But when we try to write some data on to the same file it overwrites and saves with the current data (check output)

Write a python program to open and write the content to file and read it.

```
fo=open("abc.txt","w+")
fo.write("Python Programming")
```

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type hello.txt
hi python programming
```

```
print(fo.read())
fo.close()
```

Output:

abc - Notepad

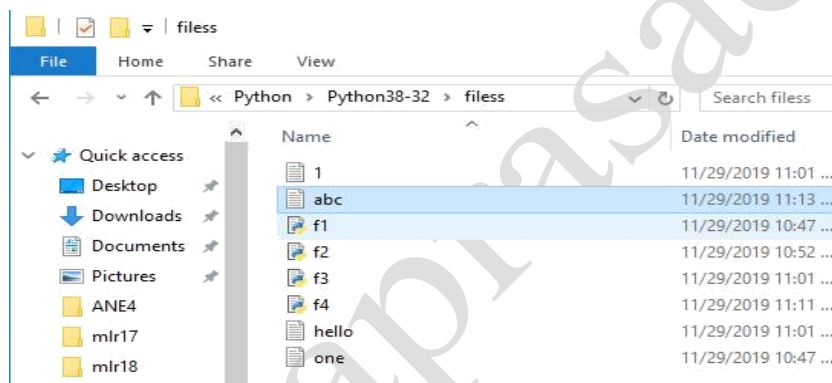
File Edit Format View Help

Python Programming introduced for III years

(or)

```
C:\Users\MRCET\AppData\Local\Programs\Python\Python38-32\filess>type abc.txt
Python Programming introduced for III years
```

Note: It creates the abc.txt file automatically and writes the data into it



Command line arguments:

The command line arguments must be given whenever we want to give the input before the start of the script, while on the other hand, `raw_input()` is used to get the input while the python program / script is running. The command line arguments in python can be processed by using either 'sys' module, 'argparse' module and 'getopt' module.

'sys' module:

Python sys module stores the command line arguments into a list, we can access it using `sys.argv`. This is very useful and simple way to read command line arguments as String.

`sys.argv` is the list of commandline arguments passed to the Python program. `argv` represents all the items that come along via the command line input, it's basically an array holding the command line arguments of our program

```
>>> sys.modules.keys() -- this prints so many dict elements in the form of list.
```

```
# Python code to demonstrate the use of 'sys' module for command line arguments
# import sys
# command line arguments are stored in the form
# of list in sys.argv
argumentList = sys.argv
```

```

    print(argumentList)
#    Print the name of file print(sys.argv[0])
#    Print the first argument after the name of file #print(sys.argv[1])

```

Output:

```

C:/Users/MIT/AppData/Local/Programs/Python/Python38-32/cmdnlinarg.py
['C:/Users/MIT/AppData/Local/Programs/Python/Python38-32/cmdnlinarg.py']
C:/Users/MIT/AppData/Local/Programs/Python/Python38-32/cmdnlinarg.py

```

Note: Since my list consist of only one element at '0' index so it prints only that list element, if we try to access at index position '1' then it shows the error like,

IndexError: list index out of range

```

import sys
print(type(sys.argv))
print('The command line arguments are:')

```

```

for i in sys.argv:
    print(i)

```

Output:
<class 'list'>

The command line arguments are:

```

#    write a python program to get python version. import sys
print("System version is:")
print(sys.version)
print("Version Information is:")
print(sys.version_info)

```

Output:

```

System version is:
3.8.0 (tags/v3.8.0:fa919fd, Oct 14 2019, 19:21:23) [MSC v.1916 32 bit (Intel)]
Version Information is:
sys.version_info(major=3, minor=8, micro=0, releaselevel='final', serial=0)

```

'argparse' module :

Python getopt module is very similar in working as the C getopt() function for parsing command-line parameters. Python getopt module is useful in parsing command line arguments where we want user to enter some options too.

```

>>> parser = argparse.ArgumentParser(description='Process some integers.')

import argparse
parser = argparse.ArgumentParser()
print(parser.parse_args())

```

'getopt' module :

Python argparse module is the preferred way to parse command line arguments. It provides a lot of option such as positional arguments, default value for arguments, help message, specifying data type of argument etc

It parses the command line options and parameter list. The signature of this function is mentioned below:

```

getopt.getopt(args, shortopts, longopts=[ ])

```

args are the arguments to be passed.

shortopts is the options this script accepts.

Optional parameter, longopts is the list of String parameters this function accepts which should be supported. Note that the -- should not be prepended with option names.

-h----- print help and usage message

-m----- accept custom option value

-d ----- run the script in debug mode

```
import getopt
```

```
import sys
```

```
argv = sys.argv[0:]
```

```
try:
```

```
    opts, args = getopt.getopt(argv, 'hm:d', ['help', 'my_file='])
```

```
    #print(opts)
```

```
    print(args)
```

```
except getopt.GetoptError:
```

```
    #Print a message or do something useful
```

```
    print('Something went wrong!')
```

```
    sys.exit(2)
```

Output:

```
C:/Users/MIT/AppData/Local/Programs/Python/Python38-32/gtopt.py ==
```

```
['C:/Users/MIT/AppData/Local/Programs/Python/Python38-32/gtopt.py']
```

Unit – 3

Object Oriented Programming: Classes and Objects; Creating Classes and Objects; Constructor Method; Classes with Multiple Objects; Objects as Arguments; Objects as Return Values; Inheritance- Single and Multiple Inheritance, Multilevel and Multipath Inheritance; Encapsulation- Definition, Private Instance Variables; Polymorphism- Definition, Operator Overloading.

GU Interface: The tkinter Module; Window and Widgets; Layout Management- pack, grid and place.

Python SQLite: The SQLite3 module; SQLite Methods- connect, cursor, execute, close; Connect to Database; Create Table; Operations on Tables- Insert, Select, Update. Delete and Drop Records.

Data Analysis: NumPy- Introduction to NumPy, Array Creation using NumPy, Operations on Arrays; Pandas- Introduction to Pandas, Series and DataFrames, Creating DataFrames from Excel Sheet and .csv file, Dictionary and Tuples. Operations on DataFrames.

Data Visualisation: Introduction to Data Visualisation; Matplotlib Library; Different Types of Charts using Pyplot- Line chart, Bar chart and Histogram and Pie chart.

Object-Oriented Programming (OOP)

Object Oriented Programming is a fundamental concept in Python, empowering developers to build modular, maintainable, and scalable applications. By understanding the core OOPs principles—classes, objects, inheritance, encapsulation, polymorphism, and abstraction—programmers can leverage the full potential of Python’s OOP capabilities to design elegant and efficient solutions to complex problems.

OOPs Concepts in Python

1. Class in Python
2. Objects in Python
3. Polymorphism in Python
4. Encapsulation in Python
5. Inheritance in Python
6. Data Abstraction in Python

1. Class in Python

A class in Python is a user-defined template for creating objects. It bundles data and functions together, making it easier to manage and use them. When we create a new class, we define a new type of object. We can then create multiple instances of this object type.

Classes are created using class keyword. Attributes are variables defined inside the class and represent the properties of the class. Attributes can be accessed using the dot . operator (e.g., MyClass.my_attribute).

2. Python Objects

In object oriented programming Python, The object is an entity that has a state and behavior associated with it. It may be any real-world object like a mouse, keyboard, chair, table, pen, etc. Integers, strings, floating-point numbers, even arrays, and dictionaries, are all objects.

An object consists of:

- **State:** It is represented by the attributes of an object.
- **Behavior:** It is represented by the methods of an object. objects.
- **Identity:** It gives a unique name to an object and enables one object to interact with other objects.

```

# Define the Animal class
class Animal:
    def __init__(self, name):
        self.name = name # Assign name attribute

    def speak(self):
        return "This is an animal." # Define a method to return a string

# Create an instance (object) of the Animal class
dog = Animal("Buddy")

# Access attributes and methods of the object
print(dog.name)
print(dog.speak())

```

Output

```

Buddy
This is an animal.

```

Create a Class

```

# define a class
class Dog:
    sound = "bark" # class attribute

```

Create Object

An Object is an instance of a Class. It represents a specific implementation of the class and holds its own data. Now, let's create an object from Dog class.

```

class Dog:
    sound = "bark"

# Create an object from the class
dog1 = Dog()

# Access the class attribute
print(dog1.sound)

```

Using __init__() Function

In Python, class has __init__() function. It automatically initializes object attributes when an object is created.

```

class Dog:
    species = "Canine" # Class attribute

    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age # Instance attribute

```

Explanation:

- class Dog: Defines a class named Dog.

- **species:** A class attribute shared by all instances of the class.
- **__init__ method:** Initializes the name and age attributes when a new object is created. Initiate Object with `__init__`

```
class Dog:
    species = "Canine" # Class attribute
    def __init__(self, name, age):
        self.name = name # Instance attribute
        self.age = age # Instance attribute
# Creating an object of the Dog class
dog1 = Dog("Buddy", 3)
print(dog1.name) # Output: Buddy
print(dog1.species) # Output: Canine
```

Output

Buddy
Canine

Explanation:

- `dog1 = Dog("Buddy", 3)`: Creates an object of the Dog class with name as "Buddy" and age as 3.
- `dog1.name`: Accesses the instance attribute name of the dog1 object.
- `dog1.species`: Accesses the class attribute species of the dog1 object.

Self Parameter

self parameter is a reference to the current instance of the class. It allows us to access the attributes and methods of the object.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def bark(self):
        print(f"{self.name} is barking!")
# Creating an instance of Dog
dog1 = Dog("Buddy", 3)
dog1.bark()
```

Output

Buddy is barking!

Explanation:

- Inside `bark()`, `self.name` accesses the specific dog's name and prints it.

- When we call `dog1.bark()`, Python automatically passes `dog1` as `self`, allowing access to its attributes.

__str__ Method

`__str__` method in Python allows us to define a custom string representation of an object. By default, when we print an object or convert it to a string using `str()`, Python uses the default implementation, which returns a string like `<__main__.ClassName object at 0x00000123>`.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def __str__(self):
        return f"{self.name} is {self.age} years old."
        # Correct: Returning a string

dog1 = Dog("Buddy", 3)
dog2 = Dog("Charlie", 5)
print(dog1)
print(dog2)
```

Output

Buddy is 3 years old.
Charlie is 5 years old.

Explanation:

- `__str__` Implementation: Defined as a method in the `Dog` class. Uses the `self` parameter to access the instance's attributes (name and age).
- Readable Output: When `print(dog1)` is called, Python automatically uses the `__str__` method to get a string representation of the object. Without `__str__`, calling `print(dog1)` would produce something like `<__main__.Dog object at 0x00000123>`.

Class and Instance Variables in Python

In Python, variables defined in a class can be either class variables or instance variables, and understanding the distinction between them is crucial for object-oriented programming.

Class Variables

These are the variables that are shared across all instances of a class. It is defined at the class level, outside any methods. All objects of the class share the same value for a class variable unless explicitly overridden in an object.

Instance Variables

Variables that are unique to each instance (object) of a class. These are defined within `__init__` method or other instance methods. Each object maintains its own copy of instance variables, independent of other objects.

Example:

```
class Dog:
    # Class variable
    species = "Canine"
    def __init__(self, name, age):
        # Instance variables
        self.name = name
        self.age = age
# Create objects
dog1 = Dog("Buddy", 3)
dog2 = Dog("Charlie", 5)
# Access class and instance variables
print(dog1.species) # (Class variable)
print(dog1.name)    # (Instance variable)
print(dog2.name)    # (Instance variable)
# Modify instance variables
dog1.name = "Max"
print(dog1.name)    # (Updated instance variable)
# Modify class variable
Dog.species = "Feline"
print(dog1.species) # (Updated class variable)
print(dog2.species)
```

Output:

```
Canine
Buddy
Charlie
Max
Feline
Feline
```

Explanation:

Class Variable (species): Shared by all instances of the class. Changing Dog.species affects all objects, as it's a property of the class itself.

Instance Variables (name, age): Defined in the __init__ method. Unique to each instance (e.g., dog1.name and dog2.name are different).

Accessing Variables: Class variables can be accessed via the class name (Dog.species) or an object (dog1.species). Instance variables are accessed via the object (dog1.name).

Updating Variables: Changing Dog.species affects all instances. Changing dog1.name only affects dog1

and does not impact dog2.

Example 2:

- A class is defined using the `class` keyword followed by the class name and a colon.
- An object is created by calling the class using parentheses.

```
class Car:
    def __init__(self, make, model):
        self.make = make
        self.model = model

    def start(self):
        print(f"Starting the {self.make} {self.model}.")

my_car = Car("Toyota", "Camry")
my_car.start() # Output: Starting the Toyota Camry.
```

Output:

```
Starting the Toyota Camry.
```

Constructor Method

- Constructor: A special method in a class that is automatically called when an object is created.
- The `__init__()` method is a special method called the constructor, which is used to initialize the object's attributes.

Classes with Multiple Objects

Multiple instances of a class can be created, each with its own attributes and methods.

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

dog1 = Dog("Buddy", 3)
dog2 = Dog("Lucy", 5)
```

Objects as Arguments and Return Values

```
def car_info(car):
    print(f"The {car.make} {car.model} is ready to go.")

car_info(my_car) # Output: The Toyota Camry is ready to go.

def create_car(make, model):
    return Car(make, model)

new_car = create_car("Ford", "Mustang")
new_car.start() # Output: Starting the Ford Mustang.
```

Output:

```
The Toyota Camry is ready to go.
Starting the Ford Mustang.
```

Polymorphism

- Polymorphism is a foundational concept in programming that allows entities like functions, methods or operators to behave differently based on the type of data they are handling. Derived from Greek, the term literally means “many forms”.
- Python’s dynamic typing and duck typing make it inherently polymorphic. Functions, operators and

even built-in objects like loops exhibit polymorphic behaviour.

- Polymorphism allows objects of different classes to be treated as objects of a common superclass.
- Operator overloading is a form of polymorphism, where operators can be redefined for custom classes.

Note: In OOP, polymorphism allows methods in different classes to share the same name but perform distinct tasks. This is achieved through inheritance and interface design. Polymorphism complements other OOP principles like inheritance (sharing behavior) and encapsulation (hiding complexity) to create robust and modular applications.

```
class Shape:
    def area(self):
        return "Undefined"

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

shapes = [Rectangle(2, 3), Circle(5)]
for shape in shapes:
    print(f"Area: {shape.area()}")
```

Explanation:

- The code showcases polymorphism using a parent class Shape and child classes Rectangle and Circle.
- **Parent Class Shape:** Contains a generic area method returning “Undefined”, acting as a placeholder for derived classes to override.
- **Child Class Rectangle:** Initializes length and width via the __init__ constructor. Overrides the area method to return the rectangle’s area as length * width.
- **Child Class Circle:** Initializes radius via the __init__ constructor. Overrides the area method to return the circle’s area as 3.14 * radius².
- **Polymorphic Behavior:** A list of shape objects (Rectangle and Circle) is created. A for loop iterates through the list, calling the area method on each object. The method executed is determined by the object’s type, showcasing polymorphism.

Inheritance

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class (called a child or derived class) to inherit attributes and methods from another class (called a parent or base class). This promotes code reuse, modularity, and a hierarchical class structure. In this article, we’ll explore inheritance in Python.

- Inheritance allows a new class to be based on an existing class, inheriting its attributes and methods.
- Python supports single and multiple inheritance, as well as multilevel and multipath inheritance.

```
class ElectricCar(Car):
    def __init__(self, make, model, battery_capacity):
        super().__init__(make, model)
        self.battery_capacity = battery_capacity
```

```

def charge(self):
    print(f"Charging the {self.make} {self.model} with a {self.battery_capacity}kWh battery.")

electric_car = ElectricCar("Tesla", "Model S", 100)
electric_car.start() # Output: Starting the Tesla Model S.
electric_car.charge() # Output: Charging the Tesla Model S with a 100kWh battery.

```

Output:

```

Starting the Tesla Model S.
Charging the Tesla Model S with a 100kWh battery.

```

Types of Python Inheritance

- **Single Inheritance:** A child class inherits from one parent class.
- **Multiple Inheritance:** A child class inherits from more than one parent class.
- **Multilevel Inheritance:** A class is derived from a class which is also derived from another class.
- **Hierarchical Inheritance:** Multiple classes inherit from a single parent class.
- **Hybrid Inheritance:** A combination of more than one type of inheritance.

Example:

1. Single Inheritance

```

class Person:
    def __init__(self, name):
        self.name = name

class Employee(Person): # Employee inherits from Person
    def __init__(self, name, salary):
        super().__init__(name)
        self.salary = salary

```

2. Multiple Inheritance

```

class Job:
    def __init__(self, salary):
        self.salary = salary

class EmployeePersonJob(Employee, Job): # Inherits from both Employee and Job
    def __init__(self, name, salary):
        Employee.__init__(self, name, salary) # Initialize Employee
        Job.__init__(self, salary) # Initialize Job

```

3. Multilevel Inheritance

```

class Manager(EmployeePersonJob): # Inherits from EmployeePersonJob
    def __init__(self, name, salary, department):
        # Explicitly initialize EmployeePersonJob

```

```
EmployeePersonJob.__init__(self, name, salary)
self.department = department
```

4. Hierarchical Inheritance

```
class AssistantManager(EmployeePersonJob): # Inherits from EmployeePersonJob
    def __init__(self, name, salary, team_size):
        # Explicitly initialize EmployeePersonJob
        EmployeePersonJob.__init__(self, name, salary)
        self.team_size = team_size
```

5. Hybrid Inheritance (Multiple + Multilevel)

```
# Inherits from both Manager and AssistantManager
class SeniorManager(Manager, AssistantManager):
    def __init__(self, name, salary, department, team_size):
        Manager.__init__(self, name, salary, department) # Initialize Manager
        # Initialize AssistantManager
        AssistantManager.__init__(self, name, salary, team_size)
```

“Creating objects to show inheritance”

Single Inheritance

```
emp = Employee("John", 40000)
print(emp.name, emp.salary)
```

Multiple Inheritance

```
emp2 = EmployeePersonJob("Alice", 50000)
print(emp2.name, emp2.salary)
```

Multilevel Inheritance

```
mgr = Manager("Bob", 60000, "HR")
print(mgr.name, mgr.salary, mgr.department)
```

Hierarchical Inheritance

```
asst_mgr = AssistantManager("Charlie", 45000, 10)
print(asst_mgr.name, asst_mgr.salary, asst_mgr.team_size)
```

Hybrid Inheritance

```
sen_mgr = SeniorManager("David", 70000, "Finance", 20)
print(sen_mgr.name, sen_mgr.salary, sen_mgr.department, sen_mgr.team_size)
```

Output

```
John 40000
Alice 50000
Bob 60000 HR
Charlie 45000 10
David 70000 Finance 20
```

Explanation:

- **Single Inheritance:** Employee inherits from Person, adding a salary attribute.
- **Multiple Inheritance:** EmployeePersonJob inherits from both Employee and Job, allowing access to both name and salary.
- **Multilevel Inheritance:** Manager inherits from EmployeePersonJob, which already includes Employee and Job.
- **Hierarchical Inheritance:** AssistantManager also inherits from EmployeePersonJob, demonstrating multiple child classes inheriting from the same parent.
- **Hybrid Inheritance:** SeniorManager inherits from both Manager (multilevel) and AssistantManager (hierarchical), combining two inheritance types.

Encapsulation

- In Python, encapsulation refers to the bundling of data (attributes) and methods (functions) that operate on the data into a single unit, typically a class. It also restricts direct access to some components, which helps protect the integrity of the data and ensures proper usage.
- Encapsulation is the concept of hiding implementation details and providing a public interface.
- Private instance variables can be defined using a double underscore prefix (__) before the variable name.

```
class PrivateClass:
    def __init__(self, value):
        self.__private_attr = value
    def __private_method(self):
        print("This is a private method.")
    def public_method(self):
        self.__private_method()
        print(f"The private attribute value is: {self.__private_attr}")
obj = PrivateClass(42)
obj.public_method()
```

Output:

```
This is a private method.
The private attribute value is: 42
```

How Encapsulation Works :

- **Data Hiding:** The variables (attributes) are kept private or protected, meaning they are not accessible directly from outside the class. Instead, they can only be accessed or modified through the methods.
- **Access through Methods:** Methods act as the interface through which external code interacts with

the data stored in the variables. For instance, getters and setters are common methods used to retrieve and update the value of a private variable.

- **Control and Security:** By encapsulating the variables and only allowing their manipulation via methods, the class can enforce rules on how the variables are accessed or modified, thus maintaining control and security over the data.

Example of Encapsulation

Encapsulation in Python is like having a bank account system where your account balance (data) is kept private. You can't directly change your balance by accessing the account database. Instead, the bank provides you with methods (functions) like deposit and withdraw to modify your balance safely.

- **Private Data (Balance):** Your balance is stored securely. Direct access from outside is not allowed, ensuring the data is protected from unauthorized changes.
- **Public Methods (Deposit and Withdraw):** These are the only ways to modify your balance. They check if your requests (like withdrawing money) follow the rules (e.g., you have enough balance) before allowing changes.
- **Public Members**
- Public members are accessible from anywhere, both inside and outside the class. These are the default members in Python.

Example:

```
class Public:
    def __init__(self):
        self.name = "John" # Public attribute
    def display_name(self):
        print(self.name) # Public method
obj = Public()
obj.display_name() # Accessible
print(obj.name) # Accessible
```

Explanation:

- **Public Attribute (name):** This attribute is declared without any underscore prefixes. It is accessible from anywhere, both inside and outside of the class.
- **Public Method (display_name):** This method is also accessible from any part of the code. It directly accesses the public attribute and prints its value.
- **Object (obj):** An instance of Public is created, and the display_name method is called, demonstrating how public attributes and methods can be accessed directly.

Note: The `__init__` method is a constructor and runs as soon as an object of a class is instantiated.

Protected members

Protected members are identified with a single underscore (`_`). They are meant to be accessed only within the class or its subclasses.

Example:

```
class Protected:
    def __init__(self):
        self._age = 30 # Protected attribute
class Subclass(Protected):
    def display_age(self):
        print(self._age) # Accessible in subclass
obj = Subclass()
obj.display_age()
```

Explanation:

- **Protected Attribute (_age):** This attribute is prefixed with a single underscore, which by convention, suggests that it should be treated as a protected member. It's not enforced by Python but indicates that it should not be accessed outside of this class and its subclasses.
- **Subclass:** Here, a subclass inherits from Protected. Within this subclass, we can still access the protected attribute _age.
- **Method (display_age):** This method within the subclass accesses the protected attribute and prints its value. This shows that protected members can be accessed within the class and its subclasses.

Private members

Private members are identified with a double underscore (__) and cannot be accessed directly from outside the class. Python uses name mangling to make private members inaccessible by renaming them internally.

Note: Python's private and protected members can be accessed outside the class through python name mangling.

```
class Private:
    def __init__(self):
        self.__salary = 50000 # Private attribute
    def salary(self):
        return self.__salary # Access through public method
obj = Private()
print(obj.salary()) # Works
#print(obj.__salary) # Raises AttributeError
```

Explanation:

- **Private Attribute (__salary):** This attribute is prefixed with two underscores, which makes it a private member. Python enforces privacy by name mangling, which means it renames the attribute in a way that makes it hard to access from outside the class.
- **Method (salary):** This public method provides the only way to access the private attribute from outside the class. It safely returns the value of __salary.

- **Direct Access Attempt:** Trying to access the private attribute directly (obj.__salary) will result in an AttributeError, showing that direct access is blocked. This is Python's way of enforcing encapsulation at a language level.

Operator Overloading

Operator Overloading means giving extended meaning beyond their predefined operational meaning. For example operator + is used to add two integers as well as join two strings and merge two lists. It is achievable because '+' operator is overloaded by int class and str class. You might have noticed that the same built-in operator or function shows different behavior for objects of different classes, this is called Operator Overloading.

Example:

```
# Python program to show use of
# + operator for different purposes.

print(1 + 2)

# concatenate two strings
print("Geeks"+"For")

# Product two numbers
print(3 * 4)

# Repeat the String
print("Geeks"*4)
```

Output

```
3
GeeksFor
12
GeeksGeeksGeeksGeeks
```

GUI Programming with Tkinter

TKinter is an open-source and standard GUI toolkit for Python. TKinter is a wrapper around tcl / TK graphical interface. TKinter is popular because of its simplicity and having a very old and active community. Also, it comes included with most binary distributions of Python. TKinter is fully portable for Macintosh, Windows, and Linux platforms. It's a good toolkit to start with since TKinter is mostly preferred for small-scale GUI applications.

Why and When Use Tkinter?

Tkinter is best used when we want to build applications with buttons, text boxes, menus and other interactive elements. Tkinter is commonly employed for making desktop applications, ranging from simple tools to more complex software. It is specially used for projects where a user friendly interface is necessary. Whether you're creating a calculator, a game, or a data visualization tool, Tkinter provides a straightforward way to design and implement GUIs in Python.

Features of Tkinter

- **Lightweight:** Tkinter is included with Python's standard library, making it lightweight and easily accessible.

- **Simple to learn:** It has a straightforward syntax, making it suitable for beginners to quickly grasp and start building GUI applications.
- **Rapid prototyping:** Ideal for rapid application development due to its simplicity and ease of use.
- **Good for basic GUIs:** Well-suited for creating simple graphical user interfaces for small to medium-sized projects.

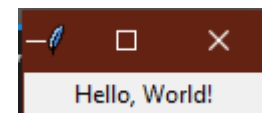
Pros of Tkinter	Cons of Tkinter
Excellent documentation	Larger memory footprint
Lightweight and bundled with Python	Limited built-in widgets
Simplicity in basic GUI development	Less modern appearance

The tkinter Module

- The tkinter module is the standard GUI library in Python, providing a set of widgets and tools for building graphical user interfaces.

```
import tkinter as tk
```

```
root = tk.Tk()
label = tk.Label(root, text="Hello, World!")
label.pack()
root.mainloop()
```



Window and Widgets

Tkinter is the standard GUI (Graphical User Interface) toolkit for Python. It provides a simple way to create windows and various widgets that allow users to interact with applications.

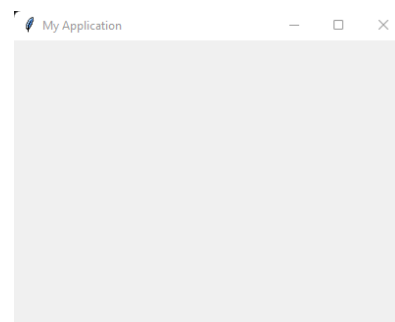
Key Concepts

1. Main Window

The main window is the primary container for all other widgets.

It is created using the Tk() class.

```
import tkinter as tk
root = tk.Tk()
root.title("My Application")
root.geometry("400x300")
root.mainloop()
```



2. Widgets

Widgets are the building blocks of a Tkinter application. Common widgets include:

- Label:** Displays text or images. Use Label() to create.
- Button:** Triggers an action when clicked. Use Button() to create.
- Entry:** A single-line text input field. Use Entry() to create.

- d. **Text:** A multi-line text input field. Use Text() to create.
- e. **Frame:** A container for organizing widgets. Use Frame() to create.

Example:

```
import tkinter as tk

def display_text():
    input_text = entry.get()
    text_area.delete(1.0, tk.END) # Clear previous text
    text_area.insert(tk.END, input_text) # Insert new text
```

Create the main window

```
root = tk.Tk()
root.title("Simple Tkinter Program")
root.geometry("400x300")
```

Create a Frame

```
frame = tk.Frame(root)
frame.pack(pady=20)
```

Label

```
label = tk.Label(frame, text="Enter some text:")
label.pack(pady=5)
```

Entry

```
entry = tk.Entry(frame, width=30)
entry.pack(pady=5)
```

Button

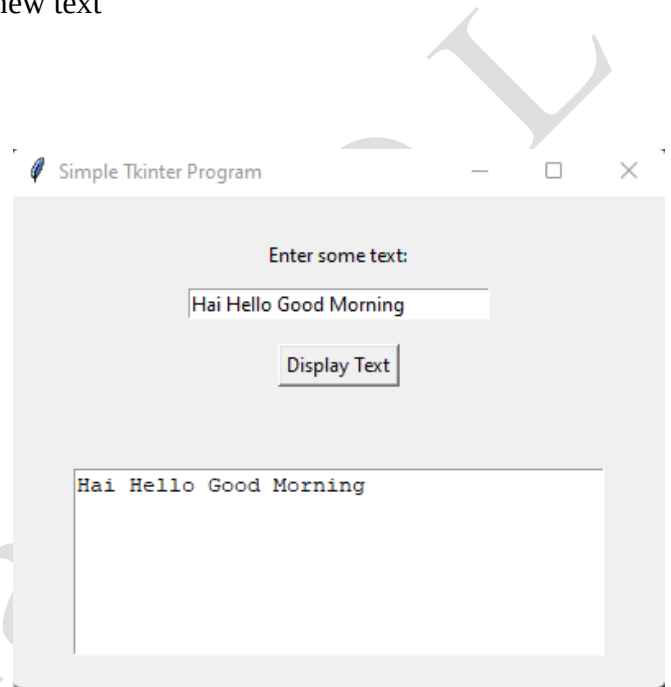
```
submit_button = tk.Button(frame, text="Display Text", command=display_text)
submit_button.pack(pady=10)
```

Text

```
text_area = tk.Text(root, height=10, width=40)
text_area.pack(pady=20)
```

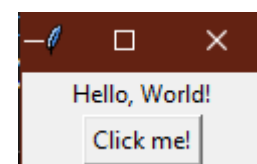
Run the application

```
root.mainloop()
```



Example 2: The Tk() function creates the main window, and various widgets (e.g., Label, Button, Entry) can be added to the window.

```
button1 = tk.Button(root, text="Click me!")
button1.pack()
```



3. Layout Management

Tkinter provides three layout managers to organize widgets:

- a) **pack()**: Packs widgets into the parent widget in a block.
- b) **grid()**: Places widgets in a 2D grid.
- c) **place()**: Places widgets at an absolute position.

Example:

```
import tkinter as tk

def show_message():
    message_label.config(text="Button Clicked!")

# Create the main window
root = tk.Tk()
root.title("Layout Managers Example")
root.geometry("400x300")

# Using pack layout manager for the title label
title_label = tk.Label(root, text="Layout Managers in Tkinter", font=("Arial", 16))
title_label.pack(pady=10)
frame = tk.Frame(root)
frame.pack(pady=20)

# Using grid layout manager for input fields and button
# Label and Entry for Name
name_label = tk.Label(frame, text="Name:")
name_label.grid(row=0, column=0, padx=5, pady=5)
name_entry = tk.Entry(frame)
name_entry.grid(row=0, column=1, padx=5, pady=5)

# Label and Entry for Age
age_label = tk.Label(frame, text="Age:")
age_label.grid(row=1, column=0, padx=5, pady=5)
age_entry = tk.Entry(frame)
age_entry.grid(row=1, column=1, padx=5, pady=5)

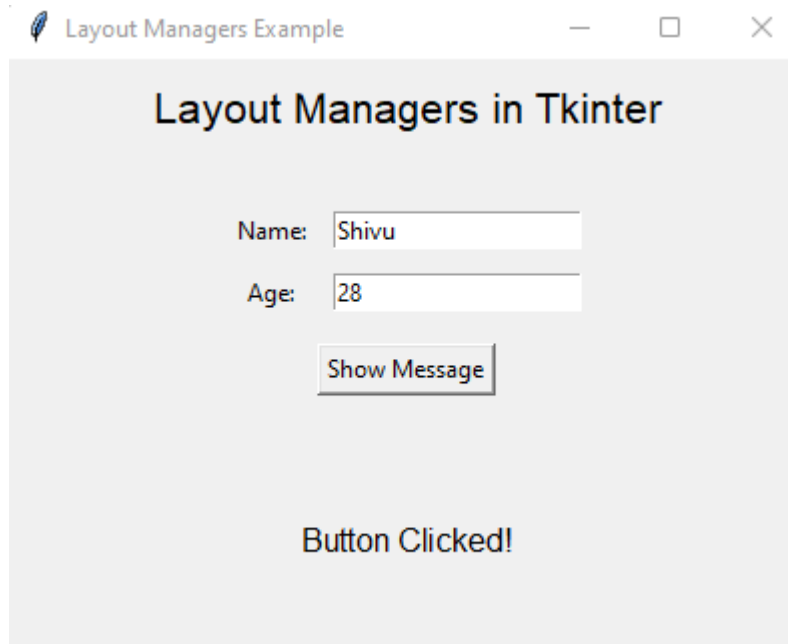
# Button to show message
show_button = tk.Button(frame, text="Show Message", command=show_message)
show_button.grid(row=2, columnspan=2, pady=10)

# Using place layout manager for a message label
message_label = tk.Label(root, text="", font=("Arial", 12))
```

```
message_label.place(relx=0.5, rely=0.8, anchor='center')
```

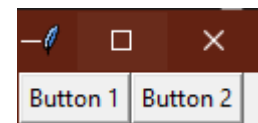
Run the application

```
root.mainloop()
```



Example 2: Tkinter provides three main layout managers: `pack()`, `grid()`, and `place()`.

```
button1 = tk.Button(root, text="Button 1")
button2 = tk.Button(root, text="Button 2")
button1.grid(row=0, column=0)
button2.grid(row=0, column=1)
```



Event Handling

Tkinter supports event-driven programming. You can bind events to widgets using the `bind()` method.

```
def on_key_press(event):
    print(f"Key pressed: {event.char}")
root.bind("<Key>", on_key_press)
```

Python SQLite

SQLite is a lightweight, serverless, self-contained SQL database engine. It is widely used for applications that require a simple database solution without the overhead of a full database server.

The SQLite3 Module

Python provides an interface to SQLite databases through the `sqlite3` module, which allows you to create, connect, and manipulate SQLite databases using Python code.

Key SQLite Methods

- **connect():** Establishes a connection to an SQLite database file.
- **cursor():** Creates a cursor object, which allows you to execute SQL commands.

- **execute():** Executes an SQL command using the cursor.
- **close():** Closes the database connection.

Connecting to a Database

To connect to a database, use the connect() method. If the database file does not exist, it will be created.

```
import sqlite3

# Connect to the database (or create it)
connection = sqlite3.connect('example.db')
```

Creating a Table

To create a table, you need to define the structure of the table and execute a CREATE TABLE SQL statement.

```
# Create a cursor object
cursor = connection.cursor()

# Create a table
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER NOT NULL
)
""")

# Commit the changes
connection.commit()
```

Operations on Tables

1. Insert Records

To insert records into a table, use the INSERT INTO SQL statement.

```
# Insert a record
cursor.execute("""
INSERT INTO users (name, age) VALUES (?, ?)
""", ('Alice', 30))

# Commit the changes
connection.commit()
```

2. Select Records

To retrieve records, use the SELECT statement.

```
# Select all records
```

```
cursor.execute('SELECT * FROM users')  
rows = cursor.fetchall()  
for row in rows:  
    print(row)
```

3. Update Records

To update existing records, use the UPDATE statement.

```
# Update a record  
cursor.execute("""  
UPDATE users SET age = ? WHERE name = ?  
""", (31, 'Alice'))  
# Commit the changes  
connection.commit()
```

4. Delete Records

To delete records from a table, use the DELETE statement.

```
# Delete a record  
cursor.execute("""  
DELETE FROM users WHERE name = ?  
""", ('Alice',))  
# Commit the changes  
connection.commit()
```

5. Drop Records

To drop a table, use the DROP TABLE statement.

```
# Drop the table  
cursor.execute("DROP TABLE IF EXISTS users")  
# Commit the changes  
connection.commit()
```

Closing the Connection

Once you are done with the database operations, it is important to close the connection.

```
# Close the cursor and connection  
cursor.close()  
connection.close()
```

Complete Example

Here's a complete example that incorporates all the above operations:

```
import sqlite3
```

```

# Connect to the database
connection = sqlite3.connect('example.db')
cursor = connection.cursor()

# Create a table
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER NOT NULL
)
""")

# Insert records
cursor.execute('INSERT INTO users (name, age) VALUES (?, ?)', ('Alice', 30))
cursor.execute('INSERT INTO users (name, age) VALUES (?, ?)', ('Bob', 25))
connection.commit()

# Select records
cursor.execute('SELECT * FROM users')
print("Users before update:")
for row in cursor.fetchall():
    print(row)

# Update a record
cursor.execute('UPDATE users SET age = ? WHERE name = ?', (31, 'Alice'))
connection.commit()

# Select records after update
cursor.execute('SELECT * FROM users')
print("Users after update:")
for row in cursor.fetchall():
    print(row)

# Delete a record
cursor.execute('DELETE FROM users WHERE name = ?', ('Bob',))
connection.commit()

# Drop the table
cursor.execute('DROP TABLE IF EXISTS users')

```

```
connection.commit()

# Close the cursor and connection

cursor.close()

connection.close()
```

Note:

- The sqlite3 module provides a simple way to interact with SQLite databases in Python.
- Common operations include connecting to a database, creating tables, inserting, selecting, updating, deleting, and dropping records.
- Always remember to commit your changes and close the connection when done.

Data Analysis with NumPy and Pandas

Data analysis involves inspecting, cleaning, and modeling data to discover useful information, inform conclusions, and support decision-making. Python provides powerful libraries such as NumPy and Pandas for data manipulation and analysis.

NumPy

Introduction to NumPy

NumPy (Numerical Python) is a fundamental library for numerical computations in Python. It provides support for arrays, matrices, and a variety of mathematical functions to operate on these data structures.

Array Creation using NumPy

You can create arrays in several ways using NumPy:

Creating a 1D Array:

```
import numpy as np
array_1d = np.array([1, 2, 3, 4, 5])
```

Creating a 2D Array:

```
array_2d = np.array([[1, 2, 3], [4, 5, 6]])
```

Using NumPy Functions:

Zeros Array:

```
zeros_array = np.zeros((2, 3)) # 2x3 array of zeros
```

Ones Array:

```
ones_array = np.ones((3, 2)) # 3x2 array of ones
```

Random Array:

```
random_array = np.random.rand(3, 3) # 3x3 array of random numbers
```

Operations on Arrays

NumPy supports a wide range of operations on arrays:

Element-wise Operations:

```
array_a = np.array([1, 2, 3])
```

```
array_b = np.array([4, 5, 6])  
sum_array = array_a + array_b # Element-wise addition
```

Mathematical Functions:

```
mean_value = np.mean(array_a) # Mean  
max_value = np.max(array_a) # Maximum
```

Array Reshaping:

```
reshaped_array = array_2d.reshape((3, 2)) # Reshape to 3x2
```

Indexing and Slicing:

```
element = array_1d[2] # Accessing the third element  
sliced_array = array_2d[:, 1] # Accessing the second column
```

Pandas

Introduction to Pandas

Pandas is a powerful data manipulation and analysis library. It provides data structures such as Series and DataFrames, which are essential for handling structured data.

Series and DataFrames

Series:

One-dimensional: It holds a single column of data.

Labeled array: Each element has an index (label).

Homogeneous: It typically holds data of a single type (e.g., all numbers, all strings).

Example: Imagine a list of names or a list of temperatures; each value has an index.

Series Creation Example:

```
import pandas as pd  
series = pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])
```

DataFrame:

Two-dimensional: It's like a spreadsheet or a table with rows and columns.

Labeled: Both rows and columns have labels (indices and column names).

Heterogeneous: It can hold multiple columns with potentially different data types.

Example: A table with columns like "Name", "Age", and "City". Each column is a Series, and the entire table is a DataFrame.

DataFrame Creation Example:

```
data = {  
    'Name': ['Alice', 'Bob', 'Charlie'],  
    'Age': [25, 30, 35],
```

```
'City': ['New York', 'Los Angeles', 'Chicago']  
}  
df = pd.DataFrame(data)
```

Creating DataFrames from Excel and CSV Files

From a CSV File:

```
df_csv = pd.read_csv('data.csv') # Load DataFrame from a CSV file
```

From an Excel Sheet:

```
df_excel = pd.read_excel('data.xlsx', sheet_name='Sheet1')  
# Load DataFrame from an Excel file
```

Creating DataFrames from Dictionary and Tuples

From a Dictionary:

```
data_dict = {  
    'Name': ['Alice', 'Bob'],  
    'Age': [25, 30] }  
df_from_dict = pd.DataFrame(data_dict)
```

From Tuples:

```
data_tuples = [('Alice', 25), ('Bob', 30)]  
df_from_tuples = pd.DataFrame(data_tuples, columns=['Name',  
    'Age'])
```

Operations on DataFrames

Viewing Data:

```
print(df.head()) # Display the first 5 rows  
print(df.tail()) # Display the last 5 rows
```

Selecting Columns:

```
ages = df['Age'] # Accessing a column
```

Filtering Rows:

```
filtered_df = df[df['Age'] > 30] # Filter rows where Age > 30
```

Adding a New Column:

```
df['Salary'] = [50000, 60000, 70000] # Adding a new column
```

Updating Values:

```
df.loc[df['Name'] == 'Alice', 'Age'] = 26 # Update Alice's age
```

Deleting a Column:

```
df.drop('Salary', axis=1, inplace=True) # Drop the Salary column
```

Group By:

```
grouped = df.groupby('City').mean()
```

```
# Group by City and calculate mean
```

Saving DataFrame to CSV:

```
df.to_csv('output.csv', index=False) # Save DataFrame to a CSV file
```

Data Visualization with Matplotlib

Data visualization is the graphical representation of information and data. By using visual elements like charts, graphs, and maps, data visualization tools provide an accessible way to see and understand trends, outliers, and patterns in data.

Introduction to Data Visualization

Purpose: To communicate data clearly and effectively through graphical means.

Benefits:

- Simplifies complex data.
- Helps identify trends and patterns.
- Aids in decision-making processes.
- Enhances data storytelling.

Matplotlib Library

Introduction to Matplotlib

Matplotlib is a widely used plotting library for Python that provides a flexible way to create a variety of static, animated, and interactive visualizations in Python.

Installation

You can install Matplotlib using pip:

```
pip install matplotlib
```

Basic Usage

To use Matplotlib, you typically import the pyplot module:

```
import matplotlib.pyplot as plt
```

Different Types of Charts using Pyplot

1. Line Chart

A line chart is used to display information as a series of data points called 'markers' connected by straight line segments.

Example:

```
import matplotlib.pyplot as plt
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

Line Chart Example



```

y = [2, 3, 5, 7, 11]

# Create a line chart

plt.plot(x, y, marker='o')

plt.title("Line Chart Example")

plt.xlabel("X-axis")

plt.ylabel("Y-axis")

plt.grid()

plt.show()

```

2. Bar Chart

A bar chart presents categorical data with rectangular bars. The lengths of the bars are proportional to the values they represent.

Example:

```

import matplotlib.pyplot as plt

# Sample data

categories = ['A', 'B', 'C', 'D']

values = [3, 7, 5, 2]

# Create a bar chart

plt.bar(categories, values, color='skyblue')

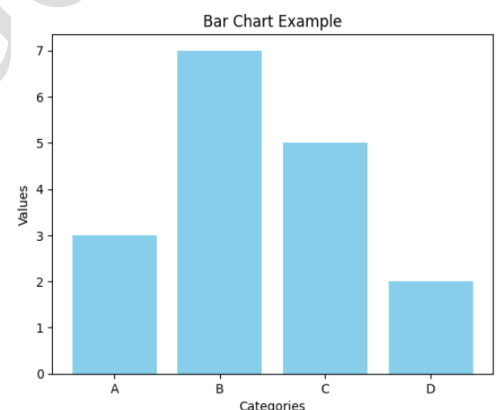
plt.title("Bar Chart Example")

plt.xlabel("Categories")

plt.ylabel("Values")

plt.show()

```



3. Histogram

A histogram is a graphical representation of the distribution of numerical data, showing the number of data points that fall within a specified range of values (bins).

Example:

```

import matplotlib.pyplot as plt

import numpy as np

# Generate random data

data = np.random.randn(1000)

# Create a histogram

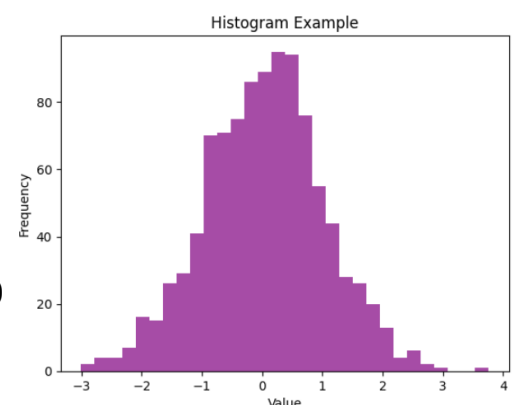
plt.hist(data, bins=30, color='purple', alpha=0.7)

plt.ylabel("Frequency")

plt.show()plt.title("Histogram Example")

plt.xlabel("Value")

```



4. Pie Chart

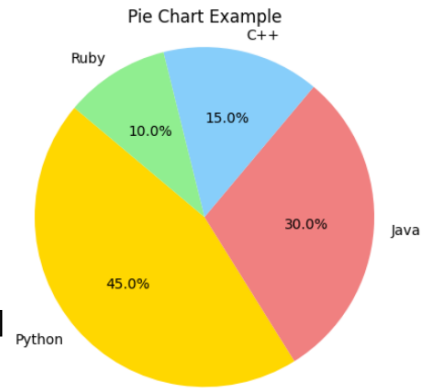
A pie chart is a circular statistical graphic divided into slices to illustrate numerical proportions.

Example:

```
import matplotlib.pyplot as plt

# Sample data
labels = ['Python', 'Java', 'C++', 'Ruby']
sizes = [45, 30, 15, 10]
colors = ['gold', 'lightcoral', 'lightskyblue', 'lightgreen']

# Create a pie chart
plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', startangle=140)
plt.title("Pie Chart Example")
plt.axis('equal') # Equal aspect ratio ensures the pie chart is circular.
plt.show()
```



Note:

Data Visualization: Essential for interpreting and presenting data effectively.

Matplotlib: A powerful library for creating a wide range of visualizations in Python.

Application of Charts:

- **Line Chart:** Ideal for displaying trends over time.
- **Bar Chart:** Suitable for comparing categorical data.
- **Histogram:** Used for showing frequency distributions.
- **Pie Chart:** Good for illustrating proportions of a whole.

Example Program:

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.

```
import matplotlib.pyplot as plt
```

Line chart

```
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.plot(x, y)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Line Chart')
plt.show()
```

Bar chart

```
plt.bar(x, y)
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Bar Chart')
plt.show()
```

Histogram

```
data = [1, 2, 3, 4, 5, 2, 3, 1, 4, 5, 3, 2, 1]
plt.hist(data, bins=5)
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title('Histogram')
plt.show()
```

Pie chart

```
labels = ['Alice', 'Bob', 'Charlie']
values = [25, 30, 35]
plt.pie(values, labels=labels, autopct='%1.1f%%')
plt.title('Pie Chart')
plt.show()
```

Output:

